

DSA MCQ BANK

200 Premium Questions for Tier-1 Engineering Interviews

Data Structures & Algorithms · Deep Code Comprehension · System Tradeoffs

Version 1.0 · Generated 2026-06-20 04:22:23

200 Questions

4 Core Areas

Tier-1 Ready

C / C++ / Java

Type	Description
Output Prediction	Dry-run code to find exact output or heap state
Bug Hunting	Find the subtle logical flaw and precise fix
Complexity Analysis	Determine strict worst/average/amortized Big-O
System Mapping	Choose optimal data structure for real-world scale

MASTER TOPIC SYLLABUS & INDEX

SECTION I — TREES & ADVANCED HIERARCHIES (Q1–Q50)

- Q1–Q2: Binary Tree Post-order & BST Height
- Q4–Q5: BFS Space & Trie Complexity
- Q7–Q8: Segment Tree Range Sum & Lazy Propagation
- Q11: Red-Black Tree Properties
- Q13: Serialization Round-Trip
- Q15: Kth Smallest in BST
- Q17: Segment Tree Build $O(n)$
- Q19: AVL Min Nodes (Fibonacci)
- Q21: Zigzag Traversal
- Q23: Segment Tree Point Update
- Q25: Count Complete Tree Nodes $O(\log^2 n)$
- Q27: Vertical Order Traversal
- Q29: Trie Pruning in Word Search II
- Q31: AVL RR Case Rotation
- Q33: BST Range Query $O(h+k)$
- Q35: Cousins in BT (depth+parent)
- Q37: BST Validation with Bounds
- Q39: Boundary Traversal Order
- Q41: Max Width BFS
- Q43: Merge Sort Tree Online vs Offline
- Q45: RB-Tree Min Nodes 2^{k-1}
- Q47: Palindrome Pairs Trie $O(nL^2)$
- Q49: Merge Sort Tree Kth $O(\log^3 n)$
- Q3: BST In-order Successor Bug
- Q6: Autocomplete Engine Design (Trie)
- Q9–Q10: AVL Rotations & Height Guarantee
- Q12: Diameter Bug ($O(n^2)$ vs $O(n)$)
- Q14: LCA Complexity
- Q16: Trie Node Memory (208 bytes)
- Q18: Morris Traversal $O(1)$ Space
- Q20: Suffix Trie vs Suffix Array
- Q22: BST Deletion Two-Children
- Q24: Path Sum Leaf Bug
- Q26: RB-Tree Recolor Rule
- Q28: Segment Tree Max Subarray
- Q30: Flatten BT to LL In-Place
- Q32: Invert BT Complexity
- Q34: Fenwick vs Segment Tree
- Q36: Compressed Trie $O(n)$ Nodes
- Q38: 2D Fenwick Tree
- Q40: AVL Deletion $O(\log n)$ Rotations
- Q42: XOR Maximization Trie $O(n)$
- Q44: Recover BST Two Swaps
- Q46: BT to DLL $O(h)$ Space
- Q48: Reconstruct BT Hash Map $O(n)$
- Q50: Threaded BT $O(1)$ Traversal

SECTION II — GRAPHS & NETWORKS (Q51–Q100)

- Q51: BFS Shortest Path Guarantee
- Q53: Dijkstra Failure on Negative Edges
- Q55: Cycle Detection via Kahn's
- Q57: DSU $\alpha(n)$ Amortized Cost
- Q59: Multi-source BFS 0-1 Matrix
- Q61: Tarjan's SCC $O(V+E)$
- Q63: Floyd-Warshall $O(V^2)$ Space
- Q65: DSU Cycle Detection
- Q67: Prim's Dense $O(V^2)$ vs Kruskal's
- Q69: DAG Shortest Path $O(V+E)$
- Q71: Offline LCA Tarjan DSU
- Q73: Word Ladder BFS Return 0
- Q75: Hamiltonian Path NP-Complete
- Q77: Max Flow Min Cut Theorem
- Q79: Path Splitting vs Halving $\alpha(n)$
- Q81: SCC Social Network Interpretation
- Q83: 8-Puzzle State Space 181440
- Q85: Kosaraju's Two-Pass SCC
- Q87: 0-1 BFS Deque $O(V+E)$
- Q89: DSU Rollback Without Path Compression
- Q52: DFS $O(V+E)$ Complexity
- Q54: Kahn's BFS Topological Sort
- Q56: Dijkstra Heap Complexity $O((V+E)\log V)$
- Q58: Kruskal's $O(E \log E)$
- Q60: Tarjan's Bridge Condition $\text{low}[v] > \text{disc}[u]$
- Q62: Bellman-Ford Negative Weights
- Q64: Bipartite Detection BFS
- Q66: Ford-Fulkerson Irrational Termination
- Q68: DFS Topological Post-order
- Q70: Articulation Points Condition
- Q72: A* Heuristic Advantage
- Q74: Euler Circuit Even Degree
- Q76: DFS Cross Edge Definition
- Q78: Rotting Oranges Multi-BFS
- Q80: Johnson's $O(V^2 \log V)$ Sparse
- Q82: Biconnected vs SCC
- Q84: DFS Grid Number of Islands
- Q86: MST Uniqueness Distinct Weights
- Q88: TSP Bitmask DP $O(2^n * n^2)$
- Q90: Dinic's $O(V^2 E)$

Q91: Shortest Cycle $O(V(V+E))$
Q93: Min-Cost Max-Flow Applications
Q95: Critical Connections = Bridge Finding
Q97: Weighted DSU Relative Ranking
Q99: Jump Game II BFS Levels

Q92: Boggle DFS+Trie $O(N^2 * 4^L)$
Q94: Alien Dictionary Topo Sort
Q96: Hopcroft-Karp $O(E \sqrt{V})$
Q98: PageRank Power Iteration
Q100: Course Schedule II Cycle=Empty

SECTION III — RECURSION & DP (Q101–Q150)

Q101: 0/1 Knapsack State $dp[i][w]$
Q103: Unbounded Knapsack Forward
Q105: LIS = LCS with Sorted Array
Q107: MCM $dp[i][j]$ Min Cost
Q109: Tower of Hanoi $2^n - 1$
Q111: LPS = LCS(s, reverse)
Q113: Subset Sum = Knapsack
Q115: Rod Cutting = Unbounded KS
Q117: LCS vs Longest Common Substring
Q119: Palindrome Partition II $dp[i]$
Q121: Distinct Subsequences DP
Q123: Catalan Numbers BST/Parens
Q125: Stock Cooldown 3 States
Q127: Palindrome Count $O(n^2)$
Q129: N-Queens 3-Set Conflict Check
Q131: Decode Ways State Machine
Q133: Distinct BSTs Catalan $C(4)=14$
Q135: Kadane's $\max(a[i], dp+a[i])$
Q137: Dice Roll $dp[d][S]$
Q139: Regex DP '*' Two Cases
Q141: K Transactions Stock $O(nk)$
Q143: Min Window Subsequence DP
Q145: Flood Fill Stack Overflow
Q147: Dice Expected Value ~ 4.0
Q149: Valid Parentheses = Catalan

Q102: Space Opt Backward Iteration
Q104: LCS 'ABCBADAB', 'BDCAB' = 4
Q106: Edit Distance Recurrence
Q108: MCM $O(n^3)$
Q110: Coin Change Greedy Fails
Q112: Wildcard Matching Correct
Q114: Memoization Sparse States
Q116: Burst Balloons Last-Burst Insight
Q118: Tree MIS DP States
Q120: Subset Sum 2^n Baseline
Q122: Largest Rectangle Stack $O(n)$
Q124: Permutations $O(n \cdot n!)$
Q126: Interleaving String 2D DP
Q128: Jump Game II Greedy $O(n)$
Q130: Longest Increasing Path $O(mn)$
Q132: Word Break DP+Trie
Q134: Memoization Space $O(nk)$
Q136: Min Path Sum Grid DP
Q138: Combination Sum Termination
Q140: Egg Drop Binary Search Opt
Q142: Bitmask vs Backtrack Subsets
Q144: Maximal Square Side-Length DP
Q146: LCS Space $O(n)$ Two Rows
Q148: Refueling Stops Max-Heap
Q150: Stock III State Machine

SECTION IV — LANGUAGE INTERNALS (Q151–Q200)

Q151: vector Capacity Doubling $\rightarrow 32$
Q153: Iterator Invalidation on Realloc
Q155: shared_ptr use_count $3 \rightarrow 2$
Q157: RAII Exception Safety unique_ptr
Q159: malloc vs calloc Zero-Init
Q161: Struct Padding 24 bytes
Q163: HashMap Treeify Threshold=8
Q165: StackOverflow Deep Recursion
Q167: volatile vs synchronized
Q169: map vs unordered_map Trade-off
Q171: Rule of Five Move Semantics
Q173: Function Pointer Syntax
Q175: Integer Cache 127 true/false
Q177: Local Variable Dangling Return
Q179: SSO Threshold 15-22 chars
Q181: GC Roots — Stack+Static+JNI

Q152: deque $O(1)$ with Larger Constant
Q154: unordered_map $O(n)$ Worst
Q156: std::move Valid-but-Unspecified
Q158: Virtual Destructor UB
Q160: realloc Null Memory Leak
Q162: Dangling Pointer Use-After-Free
Q164: String intern() $b==c$ true
Q166: ArrayList Cache Locality
Q168: Java Object Eden Space
Q170: Pointer Arithmetic 20 40
Q172: ConcurrentHashMap CAS+sync
Q174: Lambda Capture $[&x;] \rightarrow 40$
Q176: Template Code Bloat 10 fns
Q178: Java Stack vs Heap Primitives
Q180: sizeof Array Decay $\rightarrow 8$
Q182: Signed Overflow UB

Q183: String + loop $O(n^2)$
Q185: CRTP Static Polymorphism
Q187: optional `sizeof(T)+1`
Q189: Type Erasure Raw List
Q191: ThreadLocal Per-Thread Map
Q193: Placement new Pre-allocated
Q195: `getchar()` Returns int EOF
Q197: `intern()` Metaspace Risk
Q199: atomic vs mutex

Q184: Union Little-Endian $\rightarrow 4$
Q186: try-with-resources vs finalize
Q188: memmove Overlapping Regions
Q190: `constexpr` Compile-time Eval
Q192: Bit Fields `sizeof` $\rightarrow 4$
Q194: volatile Double-Checked Lock
Q196: Structured Bindings map Order
Q198: Strict Aliasing UB
Q200: JIT Inlining Threshold HotSpot

SECTION I — TREES & ADVANCED HIERARCHIES

Q1. [Easy]

[Binary Tree] — Post-order Traversal Output Prediction

Given the following binary tree and post-order traversal code, what is the printed output?

```
struct Node { int val; Node *left, *right; };
void postorder(Node* root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->val << " ";
}
// Tree: root=1, left=2(left=4,right=5), right=3(left=6,right=7)
```

- A) 1 2 3 4 5 6 7
- ✓ B) 4 5 2 6 7 3 1
- C) 4 2 5 1 6 3 7
- D) 1 2 4 5 3 6 7

Answer: B

Explanation: Post-order visits left subtree, right subtree, then root. So: 4,5,2,6,7,3,1.

Q2. [Easy]

[BST] — BST Insertion Order & Height

Keys inserted in order: 50, 30, 70, 20, 40, 60, 80. What is the height of the resulting BST?

- A) 2
- ✓ B) 3
- C) 4
- D) 7

Answer: B

Explanation: The tree is perfectly balanced with 3 levels (root=50, children=30,70, grandchildren=20,40,60,80). Height = number of edges on longest path = 2, but since height is often counted as number of nodes, it's 3.

Q3. [Medium]

[BST] — In-order Successor Bug Hunt

Find the bug in the function that returns the in-order successor of a node in a BST:

```
Node* inorderSuccessor(Node* root, Node* p) {
    Node* successor = nullptr;
    while (root) {
        if (p->val < root->val) {
            successor = root;
            root = root->left;
        } else {
            root = root->right;
        }
    }
    return successor;
}
```

A) Should use $p \rightarrow val \leq root \rightarrow val$ in condition

✓ B) The function doesn't handle the case where p has a right subtree - when $p \rightarrow val == root \rightarrow val$, it should go to $p \rightarrow right$ and find the leftmost node

C) successor should be initialized to $p \rightarrow right$

D) No bug; the function is correct

Answer: B

Explanation: The bug is that when $p \rightarrow val == root \rightarrow val$ (we found the node), the code goes right instead of finding the leftmost node in the right subtree. The correct logic should handle: if p has right subtree, successor is leftmost in right subtree; otherwise, the last ancestor where p was in left subtree.

Q4. [Easy]

[Binary Tree] — Level-order BFS Space Complexity

For a complete binary tree with n nodes, what is the worst-case space complexity of BFS level-order traversal using a queue?

A) $O(\log n)$

B) $O(n)$

✓ C) $O(n/2) = O(n)$

D) $O(1)$

Answer: C

Explanation: The last level of a complete binary tree holds $\text{ceil}(n/2)$ nodes, all of which may be in the queue simultaneously, giving $O(n)$ space.

Q5. [Easy]

[Trie] — Trie Insert & Search Complexity

A Trie stores strings over an alphabet of size Σ . What is the time complexity to insert and search a string of length L ?

- ✓ A) $O(L)$ for both insert and search
- B) $O(L * \Sigma)$
- C) $O(\log n)$ where n is number of strings
- D) $O(L^2)$

Answer: A

Explanation: Each character requires traversing one node level. Insert and search both walk L levels, so both are $O(L)$, independent of the number of stored strings.

Q6. [Medium]

[Trie] — Autocomplete Engine Design

You are building an autocomplete system that must return all words with a given prefix in under 10ms for a dictionary of 10 million words. Which data structure is most appropriate?

- A) Sorted array with binary search
- B) Hash map keyed by prefix
- ✓ C) **Compressed Trie (Patricia/Radix Trie) with DFS collection**
- D) B-Tree

Answer: C

Explanation: A compressed Trie lets you locate the prefix node in $O(L)$ and then DFS-collect all completions. A hash map would need to store every prefix explicitly. Binary search cannot group by prefix efficiently at scale.

Q7. [Easy]

[Segment Tree] — Range Sum Query Output

A segment tree is built on array $A = [1, 3, 5, 7, 9, 11]$. What does query(2, 4) (0-indexed, inclusive) return?

- A) 15
- ✓ B) **21**
- C) 19
- D) 17

Answer: B

Explanation: $A[2] + A[3] + A[4] = 5 + 7 + 9 = 21$.

Q8. [Medium]

[Segment Tree] — Lazy Propagation Purpose

Why is lazy propagation added to a segment tree?

- A) To reduce space from $O(4n)$ to $O(2n)$
- ✓ B) **To defer range updates, reducing range-update + range-query to $O(\log n)$ instead of $O(n \log n)$**
- C) To allow deletion in $O(1)$
- D) To support negative values

Answer: B

Explanation: Without lazy propagation, a range update touches $O(n)$ nodes. Lazy propagation marks nodes for deferred update, achieving $O(\log n)$ per range update.

Q9. [Medium]

[AVL Tree] — Rotation After Insertion — Left-Right Case

Insert keys 10, 5, 7 (in that order) into an empty AVL tree. Which rotations are performed?

- A) Single left rotation at root
- B) Single right rotation at root
- ✓ C) Right rotation at node 5, then left rotation at node 10
- D) No rotation needed

Answer: C

Explanation: Inserting 7 creates a Left-Right (LR) imbalance at node 10. Fix: right-rotate at 5 (making 7 the left child of 10), then left-rotate at 10. Result: 7 becomes root with 5 and 10 as children.

Q10. [Easy]

[AVL Tree] — AVL vs BST Worst-case Search

In the absolute worst case, how many comparisons does searching an AVL tree of n nodes require compared to a degenerate BST?

- A) AVL: $O(n)$, BST: $O(n)$
- ✓ B) AVL: $O(\log n)$, BST: $O(n)$
- C) AVL: $O(\log n)$, BST: $O(\log n)$
- D) AVL: $O(\sqrt{n})$, BST: $O(n)$

Answer: B

Explanation: AVL maintains a height bound of $1.44 \cdot \log_2(n)$, so search is always $O(\log n)$. A degenerate BST (all insertions in sorted order) degrades to a linked list with $O(n)$ search.

Q11. [Easy]

[Red-Black Tree] — Structural Property Enforcement

Which of the following is NOT a required property of a Red-Black Tree?

- A) Root is always black
- B) Red node's children must both be black
- ✓ C) All nodes at the same depth must have the same color
- D) Every path from a node to its NIL descendants has the same number of black nodes

Answer: C

Explanation: Red-black trees do NOT require same-depth nodes to be the same color. The actual constraints are: root is black, no consecutive red nodes, and equal black-height on every root-to-leaf path.

Q12. [Medium]

[Binary Tree] — Diameter Computation Bug

Identify the bug in this tree diameter function:

```
int height(Node* n) {
    if (!n) return 0;
    return 1 + max(height(n->left), height(n->right));
}

int diameter(Node* root) {
    if (!root) return 0;
    int lh = height(root->left);
    int rh = height(root->right);
    int throughRoot = lh + rh;
    int leftDiam = diameter(root->left);
    int rightDiam = diameter(root->right);
    return max({throughRoot, leftDiam, rightDiam});
}
```

A) height() should return -1 for null nodes

✓ B) $O(n^2)$ time — height is recomputed for every node; fix by computing height and diameter in a single DFS pass

C) throughRoot should be $lh + rh + 1$

D) Should use BFS not recursion

Answer: B

Explanation: The function is logically correct but inefficient. height() is called at every node, each call costing $O(n)$, giving $O(n^2)$ overall. The canonical fix merges both computations in one recursive pass, achieving $O(n)$.

Q13. [Easy]

[Binary Tree] — Serialize / Deserialize Round Trip

Which delimiter-based pre-order serialization correctly handles null nodes for a unique round-trip?

```
// Option A: '1,2,#,#,3,#,#' (# for null)
// Option B: '1,2,3' (only non-null)
// Option C: '1,2,#,3' (partial nulls)
// Option D: '#,1,2,3' (root last)
```

✓ A) Option A — pre-order with explicit null markers is sufficient for unique reconstruction

B) Option B — non-null values only

C) Option C — partial nulls

D) Option D — root last

Answer: A

Explanation: Pre-order with explicit null markers encodes structural information needed for unique reconstruction. Omitting nulls is ambiguous: '1,2,3' could represent multiple different trees.

Q14. [Easy]

[Binary Tree] — LCA in Binary Tree — Complexity

What is the time and space complexity of the classic recursive Lowest Common Ancestor (LCA) algorithm for a binary tree of n nodes?

- A) $O(\log n)$ time, $O(1)$ space
- ✓ B) **$O(n)$ time, $O(h)$ space where h is height**
- C) $O(n \log n)$ time, $O(n)$ space
- D) $O(h)$ time, $O(1)$ space

Answer: B

Explanation: The algorithm traverses all n nodes in the worst case (target nodes may be at leaves). Recursion stack depth is $O(h)$, the tree height ($O(n)$ worst case, $O(\log n)$ for balanced).

Q15. [Medium]

[BST] — Kth Smallest Element Strategy

What is the most efficient way to find the k -th smallest element in a BST?

- A) Store all elements in an array and sort — $O(n \log n)$
- ✓ B) **In-order traversal stopping at k -th node — $O(h + k)$**
- C) Binary search on sorted keys — $O(\log n)$
- D) BFS level-by-level — $O(n)$

Answer: B

Explanation: In-order traversal of a BST yields sorted order. Stop when the k -th node is visited. This costs $O(h)$ to reach the leftmost node plus $O(k)$ to count, totalling $O(h+k)$. With augmented subtree sizes, $O(h)$ is achievable.

Q16. [Easy]

[Trie] — Memory Overhead of Standard Trie

A Trie node for 26-letter alphabet stores an array of 26 pointers. If each pointer is 8 bytes, what is the memory per node?

- A) 26 bytes
- B) 52 bytes
- ✓ C) **208 bytes**
- D) 128 bytes

Answer: C

Explanation: 26 pointers $\times$ 8 bytes each = 208 bytes per node. This is why compressed/ternary tries are preferred for memory-constrained applications.

Q17. [Easy]

[Segment Tree] — Build Time Complexity

What is the time complexity to build a segment tree over an array of n elements?

- A) $O(n \log n)$
- ✓ B) **$O(n)$**
- C) $O(n^2)$
- D) $O(\log n)$

Answer: B

Explanation: A segment tree has at most $4n$ nodes. Building it processes each node once in a bottom-up pass, giving $O(n)$ total work.

Q18. [Hard]

[Binary Tree] — Morris Traversal Space Complexity

Morris in-order traversal achieves $O(1)$ extra space by temporarily modifying tree links. What is its time complexity?

- A) $O(n \log n)$
- ✓ B) $O(n)$
- C) $O(n^2)$
- D) $O(h)$

Answer: B

Explanation: Each edge is traversed at most twice (once to create the threaded link, once to restore it), giving $O(n)$ overall despite the non-obvious flow.

Q19. [Hard]

[AVL Tree] — Maximum Nodes at Height h

What is the minimum number of nodes in an AVL tree of height h?

- A) 2^h
- ✓ B) Fibonacci-like: $N(h) = N(h-1) + N(h-2) + 1$, with $N(0)=1, N(1)=2$
- C) h^2
- D) $2h+1$

Answer: B

Explanation: To minimize nodes at height h, one subtree has height h-1 and the other h-2 (AVL balance factor = 1). This recurrence produces Fibonacci-like growth, proving AVL height is $O(\log n)$.

Q20. [Medium]

[Trie] — Suffix Trie vs Suffix Array

For pattern matching in a text of length n, a Suffix Trie uses $O(n^2)$ space. What does a Suffix Array use?

- A) $O(n^2)$
- ✓ B) $O(n)$
- C) $O(n \log n)$
- D) $O(n * \text{alphabet_size})$

Answer: B

Explanation: A suffix array stores n integers (one per suffix) requiring $O(n)$ space, making it far more memory-efficient than a suffix trie for large texts.

Q21. [Easy]

[Binary Tree] — Zigzag Level-Order Traversal

What data structure enables zigzag (spiral) level-order traversal most naturally?

- A) Single queue
- ✓ B) Two stacks alternating direction per level
- C) Priority queue
- D) Circular buffer

Answer: B

Explanation: Two stacks allow you to push children in left-right or right-left order alternately, naturally producing zigzag output without reversing arrays.

Q22. [Easy]

[BST] — Deletion of Node with Two Children

When deleting a node with two children from a BST, what replaces it?

- A) Its left child
- B) Its right child
- ✓ C) Its in-order successor or predecessor
- D) The root

Answer: C

Explanation: The standard approach replaces the deleted node's value with its in-order successor (smallest in right subtree) or predecessor (largest in left subtree), then deletes that simpler node.

Q23. [Easy]

[Segment Tree] — Point Update Time Complexity

Given a segment tree on n elements, what is the time complexity for a single point update?

- ✓ A) $O(\log n)$
- B) $O(n)$
- C) $O(1)$
- D) $O(n \log n)$

Answer: A

Explanation: A point update walks from the leaf to the root, updating $O(\log n)$ nodes along the path.

Q24. [Medium]

[Binary Tree] — Path Sum Root-to-Leaf Bug

What is the bug in this path-sum function?

```
bool hasPathSum(Node* root, int target) {  
    if (!root) return target == 0;  
    return hasPathSum(root->left, target - root->val) ||  
        hasPathSum(root->right, target - root->val);  
}
```

- ✓ A) Returns true for internal nodes when remaining sum is 0 — must check both children are null before returning true
- B) Should subtract root->val before checking null
- C) Should use && not ||
- D) No bug

Answer: A

Explanation: If an internal node's remaining sum becomes 0, the null check on its children returns true even though we haven't reached a leaf. Fix: if(!root->left && !root->right) return target == root->val; at the leaf check.

Q25. [Hard]

[Binary Tree] — Count Complete Tree Nodes Efficiently

What is the best time complexity to count nodes in a complete binary tree?

- A) $O(n)$
- B) $O(\log n)$
- ✓ C) $O(\log^2 n)$
- D) $O(n \log n)$

Answer: C

Explanation: Compare the height of leftmost and rightmost paths: if equal, the tree is a perfect binary tree and has $2^h - 1$ nodes. Otherwise recurse on subtrees. Each comparison is $O(\log n)$, and recursion depth is $O(\log n)$, giving $O(\log^2 n)$.

Q26. [Medium]

[Red-Black Tree] — Insertion Color Fix Rule

During Red-Black Tree insertion, if the uncle of the newly inserted node is RED, what is the fix?

- ✓ A) Recolor parent and uncle to black, grandparent to red, then re-check grandparent
- B) Perform a single rotation at the grandparent
- C) Perform a double rotation
- D) Delete the uncle

Answer: A

Explanation: When uncle is red, no structural rotation is needed — simply recolor parent and uncle black, grandparent red, and propagate the fix upward from the grandparent.

Q27. [Medium]

[Binary Tree] — Vertical Order Traversal

For vertical order traversal of a binary tree, which combination of data structures gives $O(n \log n)$ time?

- A) Stack + array
- ✓ B) Map keyed by horizontal distance + BFS queue
- C) Two queues
- D) DFS + linked list

Answer: B

Explanation: Assign horizontal distance (hd) to each node: root=0, left child=hd-1, right child=hd+1. Use BFS to process level-by-level. A sorted map on hd collects nodes per column. Total: $O(n \log n)$ due to map insertions.

Q28. [Hard]

[Segment Tree] — Maximum Subarray with Segment Tree

A segment tree node storing (maxPrefix, maxSuffix, maxSum, total) for maximum subarray queries requires how many merging operations per query?

- A) $O(1)$
- ✓ B) $O(\log n)$ merge steps, each $O(1)$
- C) $O(n)$
- D) $O(\log^2 n)$

Answer: B

Explanation: Each node aggregation is $O(1)$. A query touches $O(\log n)$ nodes, each merged in $O(1)$, giving $O(\log n)$ total.

Q29. [Medium]

[Trie] — Word Search II — Trie vs Brute Force

In the 'Word Search II' problem (find all words from a list on a 2D board), using a Trie instead of hashing each word gives what improvement?

- A) No improvement
- ✓ B) Allows pruning search branches when no word shares the current prefix, reducing backtracking
- C) Reduces space to $O(1)$
- D) Allows parallel search

Answer: B

Explanation: With a Trie, during DFS on the board we can immediately stop a path if the current character sequence doesn't match any Trie node, pruning invalid branches early — a significant practical speedup.

Q30. [Hard]

[Binary Tree] — Flatten Binary Tree to Linked List

Flattening a binary tree to a linked list in-place (pre-order) in $O(n)$ time with $O(1)$ extra space uses which technique?

- A) BFS with queue
- B) Extra stack for pre-order
- ✓ C) Reverse post-order (right, left, root) with a 'prev' pointer
- D) Morris traversal

Answer: C

Explanation: By processing nodes in reverse pre-order (right subtree first, then left, then root), we can set $node \rightarrow right = prev$ and null $node \rightarrow left$, linking nodes in pre-order from tail to head without extra space.

Q31. [Easy]

[AVL Tree] — Right-Right Case Rotation

Keys inserted into an empty AVL tree: 10, 20, 30. What single rotation fixes the imbalance?

- ✓ A) Left rotation at node 10 (RR case)
- B) Right rotation at node 20
- C) Left-Right double rotation
- D) No rotation needed

Answer: A

Explanation: Inserting 30 causes an RR imbalance at node 10 (balance factor = -2). A single left rotation at 10 makes 20 the new subtree root with 10 as left child and 30 as right child.

Q32. [Easy]

[Binary Tree] — Invert Binary Tree Complexity

What are the time and space complexities of recursively inverting a binary tree?

- A) $O(n)$ time, $O(1)$ space
- ✓ B) $O(n)$ time, $O(h)$ space
- C) $O(n \log n)$ time, $O(h)$ space
- D) $O(h)$ time, $O(h)$ space

Answer: B

Explanation: Every node is visited once for $O(n)$ time. The call stack depth equals the tree height h , giving $O(h)$ space ($O(n)$ worst case for skewed trees, $O(\log n)$ for balanced).

Q33. [Medium]

[BST] — Range Query in BST

Find all keys in BST in range $[L, R]$. What is the time complexity?

- A) $O(n)$
- ✓ B) $O(h + k)$ where k is the number of keys in range
- C) $O(\log n)$
- D) $O(n \log n)$

Answer: B

Explanation: Navigate to L in $O(h)$ time, then in-order traverse collecting k keys in $O(k)$. Total: $O(h+k)$. For a balanced BST, $h=O(\log n)$.

Q34. [Medium]

[Segment Tree] — Fenwick Tree vs Segment Tree

For range sum queries and point updates only, why is a Fenwick Tree (BIT) preferred over a Segment Tree?

- A) Fenwick trees support range updates natively
- ✓ B) Fenwick trees have $O(\log n)$ operations with smaller constants and simpler $O(n)$ implementation
- C) Fenwick trees have $O(1)$ query
- D) Fenwick trees require less pre-computation

Answer: B

Explanation: Both structures achieve $O(\log n)$ per operation, but a Fenwick tree's index arithmetic is simpler, cache-friendlier, and requires roughly half the code and memory of a segment tree for this restricted use case.

Q35. [Easy]

[Binary Tree] — Cousins in Binary Tree

Two nodes are cousins if they are at the same depth but have different parents. What BFS metadata suffices to check this?

- A) Only depth
- B) Only parent
- ✓ C) Both depth and parent for each node
- D) Neither — DFS is required

Answer: C

Explanation: Nodes are cousins iff $\text{depth}(a) == \text{depth}(b)$ AND $\text{parent}(a) \neq \text{parent}(b)$. BFS naturally tracks depth (level); you augment with parent references to distinguish cousins from siblings.

Q36. [Hard]

[Trie] — Compressed Trie Space Advantage

A compressed (Patricia) Trie for n keys with average key length L reduces node count from $O(n \cdot L)$ to what?

- A) $O(n \cdot L)$
- ✓ B) $O(n)$
- C) $O(L)$
- D) $O(n + L)$

Answer: B

Explanation: A Patricia Trie merges chains of single-child nodes into single edges. The number of internal nodes is at most $n-1$ and the number of leaves is n , giving $O(n)$ nodes total regardless of key length.

Q37. [Easy]

[Binary Tree] — BST Validation Using Range

The correct approach to validate a BST uses:

```
bool isValid(Node* n, long minVal, long maxVal) {
    if (!n) return true;
    if (n->val <= minVal || n->val >= maxVal) return false;
    return isValid(n->left, minVal, n->val) &&
        isValid(n->right, n->val, maxVal);
}
```

- A) Only comparing parent and child values
- ✓ B) Passing min/max bounds down the recursion — shown above
- C) In-order traversal and checking sorted order
- D) Both B and C are equally correct

Answer: B

Explanation: Option C (in-order check) is also valid but Option B is the canonical recursive approach shown. Only the range-propagation correctly enforces that all left-subtree values are less than the root, not just immediate children.

Q38. [Hard]

[Segment Tree] — 2D Segment Tree Use Case

You need range sum queries on a 2D matrix where point updates occur frequently. Which structure is most appropriate?

- A) 1D Segment Tree with row merging
- ✓ B) 2D Fenwick Tree (BIT)
- C) Sparse Table
- D) Trie

Answer: B

*Explanation: A 2D Fenwick Tree supports $O(\log M * \log N)$ point update and range sum query for an $M \times N$ matrix with $O(M*N)$ space — better constant factors than a 2D segment tree for this specific use case.*

Q39. [Medium]

[Binary Tree] — Boundary Traversal

Boundary traversal of a binary tree visits nodes in which order?

- A) BFS order
- ✓ B) Left boundary (top-down), leaf nodes (left-right), right boundary (bottom-up)
- C) Pre-order DFS
- D) Post-order DFS

Answer: B

Explanation: Boundary traversal: (1) left boundary excluding leaves top-to-bottom, (2) all leaf nodes left-to-right, (3) right boundary excluding leaves bottom-to-top. This traces the outer perimeter of the tree.

Q40. [Hard]**[AVL Tree] — Deletion Rotations**

After deleting a node from an AVL tree, how many rotations may be required to restore balance?

- A) At most 1
- B) At most 2
- ✓ C) $O(\log n)$ — rotations may propagate up the entire tree
- D) At most 3

Answer: C

Explanation: Unlike insertion which requires at most 2 rotations, AVL deletion may require rebalancing at every ancestor up to the root, totalling $O(\log n)$ rotations.

Q41. [Easy]**[Binary Tree] — Maximum Width of Binary Tree**

Maximum width (maximum number of nodes at any level) of a binary tree can be found using BFS in what complexity?

- ✓ A) $O(n)$ time, $O(n)$ space
- B) $O(n \log n)$ time, $O(1)$ space
- C) $O(n)$ time, $O(1)$ space
- D) $O(\log n)$ time

Answer: A

Explanation: BFS processes all n nodes once ($O(n)$ time). The queue holds at most the widest level, which can be $O(n/2) = O(n)$ nodes.

Q42. [Medium]**[Trie] — XOR Maximization with Trie**

To find the maximum XOR of two numbers in an array, a bit-level Trie is built and queried. What is the time complexity?

- A) $O(n^2)$
- ✓ B) $O(32n) = O(n)$
- C) $O(n \log n)$
- D) $O(n^2 \log n)$

Answer: B

Explanation: Building the Trie: $O(32n)$ insertions, each $O(32)$. Querying for each element: $O(32n)$. Total: $O(n)$ since 32 is the constant bit width of integers.

Q43. [Hard]

[Segment Tree] — Offline vs Online Range Queries

Merge Sort Tree supports online (arbitrary order) range queries in $O(\log^2 n)$. What does it sacrifice vs offline approaches?

- A) Correctness
- ✓ B) Time constant — offline approaches (e.g., persistent segment tree) can achieve $O(\log n)$ per query
- C) Space
- D) It can only handle sorted input

Answer: B

Explanation: Offline methods like persistence allow $O(\log n)$ queries by reusing immutable structure versions. Merge Sort Tree is online (answers queries as they arrive) but pays $O(\log^2 n)$.

Q44. [Hard]

[Binary Tree] — Recover BST with Two Swapped Nodes

Two nodes in a BST are accidentally swapped. In-order traversal will reveal them as:

- A) The first and last elements
- ✓ B) Either one or two inversions (adjacent or non-adjacent swap)
- C) The root and a leaf
- D) Cannot be detected by in-order

Answer: B

Explanation: If swapped nodes are adjacent in in-order, there is one inversion (first > second). If non-adjacent, there are two inversions. The first element of the first inversion and the second element of the last inversion are the swapped nodes.

Q45. [Hard]

[Red-Black Tree] — Black Height Guarantee

A Red-Black Tree with black-height k has at least how many internal nodes?

- A) k
- B) $2k$
- C) $2k - 1$
- ✓ D) $2^k - 1$

Answer: D

Explanation: In a tree with black-height k , if all nodes were black (minimum nodes), it would be a perfect binary tree with height k and $2^k - 1$ nodes.

Q46. [Medium]

[Binary Tree] — Binary Tree to DLL Conversion

Converting a binary tree to a doubly linked list in-place using in-order traversal requires what extra space?

- A) $O(n)$
- ✓ B) $O(h)$ — recursion stack only
- C) $O(1)$
- D) $O(n \log n)$

Answer: B

Explanation: The conversion is done during in-order traversal, which uses $O(h)$ stack space for recursion. No additional data structures are needed.

Q47. [Hard]

[Trie] — Palindrome Pairs Using Trie

For the 'Palindrome Pairs' problem with n words, using a Trie achieves what complexity vs brute force?

- A) Both are $O(n^2 * L)$
- ✓ B) Trie: $O(n * L^2)$, Brute Force: $O(n^2 * L)$
- C) Trie: $O(n * L)$, Brute Force: $O(n^2)$
- D) Trie: $O(n * L * \log n)$, Brute Force: $O(n^2 * L)$

Answer: B

*Explanation: With a Trie, for each word of length L , we check all $O(L)$ prefixes and suffixes for palindrome conditions in $O(L)$ each, giving $O(L^2)$ per word and $O(n * L^2)$ total. Brute force checks all n^2 pairs, each in $O(L)$.*

Q48. [Medium]

[Binary Tree] — Construct Tree from Inorder and Preorder

Reconstructing a binary tree from inorder + preorder traversals. Using a hash map for inorder indices reduces complexity from $O(n^2)$ to?

- A) $O(n \log n)$
- ✓ B) $O(n)$
- C) $O(n \sqrt{n})$
- D) $O(\log n)$

Answer: B

Explanation: With a hash map, each preorder node lookup in the inorder array is $O(1)$. Processing n nodes with $O(1)$ each gives $O(n)$ total.

Q49. [Hard]

[Segment Tree] — Merge Sort Tree Range Kth Smallest

A Merge Sort Tree can answer 'k-th smallest in range $[l, r]$ ' in what time?

- A) $O(n)$
- B) $O(\log n)$
- ✓ C) $O(\log^3 n)$ with binary search + fractional cascading
- D) $O(n \log n)$

Answer: C

Explanation: Binary search on the answer value, with each check requiring $O(\log^2 n)$ time in the Merge Sort Tree, gives $O(\log^3 n)$. With fractional cascading, it reduces to $O(\log^2 n)$.

Q50. [Medium]

[Binary Tree] — Threaded Binary Tree Advantage

A threaded binary tree's primary advantage is:

- A) Faster insertion
- B) Reduced height
- ✓ C) In-order traversal without stack or recursion ($O(1)$ space)
- D) Automatic balancing

Answer: C

Explanation: Threaded trees use otherwise-null right (and sometimes left) pointers to point to the in-order successor (or predecessor). This enables stack-free, recursion-free in-order traversal in $O(1)$ extra space.

SECTION II — GRAPHS & NETWORKS

Q51. [Easy]

[BFS] — Shortest Path in Unweighted Graph

Why does BFS guarantee shortest path in an unweighted, undirected graph?

- A) BFS visits nodes alphabetically
- B) BFS uses a priority queue
- ✓ C) **BFS expands nodes in non-decreasing order of edge count from source**
- D) BFS avoids cycles

Answer: C

Explanation: BFS explores all nodes at distance d before any node at distance $d+1$. The first time a node is discovered, it is via the fewest edges, which equals the shortest path in unweighted graphs.

Q52. [Easy]

[DFS] — DFS Time Complexity on Sparse Graphs

For a directed graph with V vertices and E edges, DFS runs in:

- A) $O(V^2)$
- ✓ B) **$O(V + E)$**
- C) $O(E \log V)$
- D) $O(V * E)$

Answer: B

Explanation: DFS visits each vertex once ($O(V)$) and traverses each edge once ($O(E)$), giving $O(V+E)$. For sparse graphs ($E \ll V^2$), this is much better than $O(V^2)$ matrix-based approaches.

Q53. [Medium]

[Dijkstra] — Dijkstra with Negative Edges

Why does Dijkstra's algorithm fail on graphs with negative edge weights?

- A) It cannot handle directed graphs
- ✓ B) **Once a node is marked visited with tentative distance d , a later negative edge could yield a shorter path, which Dijkstra ignores**
- C) The priority queue becomes corrupted
- D) It always gives the wrong answer on negative graphs

Answer: B

Explanation: Dijkstra's greedy choice assumes that the minimum tentative distance is final. Negative edges invalidate this: a longer path with a negative edge can beat a shorter path with positive edges after the node is 'settled'.

Q54. [Easy]

[Topological Sort] — Kahn's Algorithm — BFS Topological Sort

Kahn's algorithm for topological sort uses:

- A) DFS with back-edge tracking
- ✓ **B) In-degree array + queue processing zero-in-degree nodes**
- C) Priority queue on node weights
- D) Union-Find

Answer: B

Explanation: Kahn's: initialize a queue with all nodes of in-degree 0. Repeatedly dequeue a node, add it to the result, and decrement in-degrees of its neighbors; enqueue any that reach 0. If result size < V, a cycle exists.

Q55. [Easy]

[Topological Sort] — Detecting Cycle with Topological Sort

In Kahn's algorithm, if the topological sort produces fewer than V vertices, what does it indicate?

- A) Graph is disconnected
- ✓ **B) Graph contains a cycle**
- C) Graph has isolated vertices
- D) In-degree was computed incorrectly

Answer: B

Explanation: Nodes in a cycle never reach in-degree 0 (each is always waiting for another in-cycle node to be removed). Fewer than V nodes in the output means at least one cycle exists.

Q56. [Medium]

[Dijkstra] — Dijkstra Priority Queue Implementation

Using a binary heap (priority_queue), Dijkstra's algorithm on V vertices and E edges runs in:

- A) $O(V^2)$
- ✓ **B) $O((V + E) \log V)$**
- C) $O(E \log E)$
- D) $O(V \log V + E)$

Answer: B

Explanation: Each vertex is extracted from the heap once: $O(V \log V)$. Each edge may trigger a relaxation/heap push: $O(E \log V)$. Total: $O((V+E) \log V)$. With a Fibonacci heap, this reduces to $O(E + V \log V)$.

Q57. [Hard]

[DSU] — Union by Rank + Path Compression Amortized Cost

With both union by rank and path compression, the amortized cost per DSU operation is:

- A) $O(\log n)$
- B) $O(\log \log n)$
- ✓ **C) $O(\alpha(n))$ — inverse Ackermann, effectively $O(1)$**
- D) $O(1)$ exactly

Answer: C

Explanation: The inverse Ackermann function $\alpha(n)$ grows so slowly that $\alpha(n) \leq 4$ for all practical n ($n < 10^{80000}$). Combined union-by-rank and path compression achieves this near-constant amortized complexity.

Q58. [Medium]

[DSU] — Kruskal's MST with DSU

Kruskal's Minimum Spanning Tree algorithm sorts edges and uses DSU. Its total time complexity is dominated by:

- ✓ A) $O(E \log E)$ for sorting edges
- B) $O(V^2)$
- C) $O(E * \alpha(V))$ for union-find operations
- D) $O(V \log V)$

Answer: A

*Explanation: Sorting E edges takes $O(E \log E)$. The union-find operations for all edges take $O(E * \alpha(V)) \approx O(E)$. Sorting dominates, giving $O(E \log E)$ overall.*

Q59. [Medium]

[BFS] — Multi-source BFS for 0-1 Matrix

In the '01 Matrix' problem (distance to nearest 0 for each cell), multi-source BFS initializes the queue with:

- A) All cells with value 1
- ✓ B) All cells with value 0 simultaneously
- C) The cell (0,0) only
- D) Cells with maximum value

Answer: B

*Explanation: Multi-source BFS starts all '0' cells at distance 0 simultaneously. BFS then naturally propagates minimum distances to '1' cells, correctly computing the nearest-0 distance for every cell in $O(m*n)$.*

Q60. [Hard]

[Tarjan] — Tarjan's Bridge-Finding — Key Property

In Tarjan's bridge-finding algorithm, an edge (u,v) is a bridge if:

- A) $disc[u] < disc[v]$
- B) $low[v] == disc[u]$
- ✓ C) $low[v] > disc[u]$
- D) $low[u] < disc[v]$

Answer: C

Explanation: $low[v] > disc[u]$ means no back-edge from the subtree rooted at v can reach u or its ancestors, so removing edge (u,v) disconnects the graph — it's a bridge.

Q61. [Hard]

[Tarjan] — Tarjan's SCC Algorithm Time Complexity

Tarjan's Strongly Connected Components algorithm runs in:

- A) $O(V^2 + E)$
- ✓ B) $O(V + E)$
- C) $O(V \log V + E)$
- D) $O(E \log V)$

Answer: B

Explanation: Tarjan's SCC does a single DFS pass maintaining a stack and low-link values. Each vertex and edge is processed once: $O(V+E)$.

Q62. [Medium]

[Graphs] — Bellman-Ford vs Dijkstra Trade-off

Bellman-Ford is preferred over Dijkstra when:

- A) The graph has no negative cycles
- ✓ **B) The graph has negative edge weights (and you want to detect negative cycles)**
- C) The graph is dense
- D) V is very large

Answer: B

Explanation: Bellman-Ford handles negative weights and detects negative cycles by running V-1 relaxation passes. Its $O(VE)$ complexity is worse than Dijkstra's $O((V+E) \log V)$, but it's correct for negative-weight graphs.

Q63. [Easy]

[Graphs] — Floyd-Warshall Space Complexity

Floyd-Warshall all-pairs shortest path uses how much space?

- A) $O(V)$
- B) $O(E)$
- ✓ **C) $O(V^2)$**
- D) $O(V^3)$

Answer: C

Explanation: Floyd-Warshall maintains a $V \times V$ distance matrix (and optionally a predecessor matrix), requiring $O(V^2)$ space. The algorithm itself runs in $O(V^3)$ time.

Q64. [Easy]

[BFS] — Bipartite Graph Detection

BFS-based bipartite graph detection assigns two colors to nodes. The graph is bipartite if and only if:

- A) All nodes of the same color are connected
- ✓ **B) No edge connects two nodes of the same color**
- C) The graph is connected
- D) There are exactly two connected components

Answer: B

Explanation: A graph is bipartite if vertices can be 2-colored such that no edge connects same-colored vertices. BFS colors each neighbor opposite to the current node; if a conflict is found, the graph is not bipartite.

Q65. [Easy]

[DSU] — Detecting Cycle in Undirected Graph with DSU

Using DSU to detect a cycle in an undirected graph: a cycle exists when?

- A) Two vertices have the same rank
- ✓ **B) An edge (u,v) is found where $\text{find}(u) == \text{find}(v)$**
- C) The union operation returns false
- D) Any component has more edges than vertices

Answer: B

Explanation: If $\text{find}(u) == \text{find}(v)$ before union, nodes u and v are already in the same component. Adding edge (u,v) would create a cycle.

Q66. [Hard]

[Graphs] — Network Flow — Ford-Fulkerson Termination

Ford-Fulkerson may not terminate if edge capacities are:

- A) Very large integers
- B) Integers
- ✓ C) Irrational numbers
- D) All equal to 1

Answer: C

Explanation: With irrational capacities, augmenting paths may yield augmenting flows that sum to a series that doesn't converge to the max flow. Edmonds-Karp (BFS-based) guarantees termination regardless.

Q67. [Medium]

[Graphs] — Prim's vs Kruskal's — Dense vs Sparse

For a dense graph ($E \approx V^2$), which MST algorithm is preferred?

- ✓ A) Prim's with adjacency matrix — $O(V^2)$
- B) Kruskal's — $O(E \log E) = O(V^2 \log V)$
- C) Bellman-Ford
- D) Floyd-Warshall

Answer: A

Explanation: For dense graphs, Prim's with an adjacency matrix runs in $O(V^2)$, better than Kruskal's $O(E \log E) = O(V^2 \log V)$. For sparse graphs, Kruskal's with DSU is preferred.

Q68. [Easy]

[DFS] — Topological Sort via DFS

In DFS-based topological sort, vertices are added to the result in:

- A) Pre-order (when first visited)
- ✓ B) Post-order reversed (when DFS finishes a vertex)
- C) BFS order
- D) Alphabetical order

Answer: B

Explanation: A vertex is pushed onto a stack when its DFS finishes (all descendants are processed). Reversing this post-order gives a valid topological ordering.

Q69. [Medium]

[Graphs] — Shortest Path in DAG

For a DAG, the most efficient shortest path algorithm is:

- A) Dijkstra's
- B) Bellman-Ford
- ✓ C) Topological sort + single relaxation pass in $O(V+E)$
- D) Floyd-Warshall

Answer: C

Explanation: In a DAG, topological ordering ensures each vertex is relaxed after all predecessors. One pass in topological order relaxes all edges correctly, with $O(V+E)$ complexity and no need for negative-edge handling.

Q70. [Hard]

[Tarjan] — Articulation Points vs Bridges

What is the condition for a vertex u to be an articulation point in Tarjan's algorithm?

- A) $\text{low}[v] > \text{disc}[u]$ for all children v
- ✓ B) u is root with 2+ children, OR u is non-root and has a child v with $\text{low}[v] \geq \text{disc}[u]$
- C) u has the highest disc value
- D) u is connected to every other vertex

Answer: B

Explanation: For non-root u : child v with $\text{low}[v] \geq \text{disc}[u]$ means no back-edge from v 's subtree goes above u , so removing u disconnects v . For root: it's an articulation point iff it has 2+ children in the DFS tree.

Q71. [Hard]

[DSU] — Offline LCA with DSU (Tarjan's LCA)

Tarjan's offline LCA algorithm processes queries in what order?

- A) BFS order
- ✓ B) DFS post-order with union-find to track ancestors
- C) Sorted by depth
- D) Random order

Answer: B

Explanation: Tarjan's offline LCA does DFS. When a node u finishes, it's unioned with its parent. When processing a query (u, v) : if v is already visited, $\text{LCA}(u, v) = \text{find}(v)$. All queries must be known upfront (offline).

Q72. [Medium]

[Graphs] — A* vs Dijkstra

A* outperforms Dijkstra by:

- A) Using a Fibonacci heap
- ✓ B) Using a heuristic to prioritize exploration toward the goal, reducing nodes examined
- C) Running in $O(V)$ instead of $O(V \log V)$
- D) Handling negative edges

Answer: B

Explanation: A uses $f(n) = g(n) + h(n)$ where h is an admissible heuristic estimating cost to goal. This focuses the search, potentially examining far fewer nodes than Dijkstra's uniform expansion.*

Q73. [Medium]

[BFS] — Word Ladder — BFS Level Count

In the Word Ladder problem, BFS gives the minimum transformation sequence. If no sequence exists, BFS returns:

- A) -1
- B) The length of beginWord
- ✓ C) 0 (no path found)
- D) Infinity

Answer: C

Explanation: By convention (LeetCode definition), if no transformation sequence exists, return 0. BFS naturally handles this: if the target is never reached, return 0 after exhausting all possibilities.

Q74. [Easy]

[Graphs] — Euler Circuit Condition

A connected undirected graph has an Euler circuit (visits every edge exactly once, returns to start) iff:

- A) Every vertex has degree > 1
- ✓ **B) Every vertex has even degree**
- C) The graph is bipartite
- D) There are no cycles

Answer: B

Explanation: Euler's theorem: an Euler circuit exists iff the graph is connected and every vertex has even degree. If exactly 2 vertices have odd degree, an Euler path (not circuit) exists between them.

Q75. [Hard]

[Graphs] — Hamiltonian Path NP-Completeness

Unlike Euler paths, Hamiltonian paths (visit every vertex exactly once) are:

- A) Solvable in $O(V + E)$ with DFS
- ✓ **B) NP-complete — no known polynomial algorithm**
- C) Solvable with topological sort
- D) Solvable with Dijkstra's

Answer: B

Explanation: The Hamiltonian path problem is NP-complete. No polynomial algorithm is known. The best exact algorithms are exponential ($O(2^n \cdot n)$ with bitmask DP). Contrast with Euler paths which are $O(E)$.

Q76. [Hard]

[DFS] — DFS Edge Classification

In DFS on a directed graph, a 'cross edge' is:

- A) An edge to an ancestor in the DFS tree
- B) An edge within the same DFS tree
- ✓ **C) An edge from one DFS subtree to another (or a different DFS tree component)**
- D) An edge that creates a cycle

Answer: C

Explanation: DFS classifies edges as: tree edges (in DFS tree), back edges (to ancestor, indicating cycle), forward edges (to descendant, directed only), cross edges (between unrelated subtrees or components, directed only).

Q77. [Hard]

[Graphs] — Max Flow Min Cut Theorem

The Max-Flow Min-Cut theorem states:

- A) Maximum flow equals the sum of all edge capacities
- ✓ **B) Maximum flow in a network equals the minimum capacity cut separating source from sink**
- C) Min cut is always unique
- D) Max flow can be computed in $O(V)$ time

Answer: B

Explanation: Max-Flow Min-Cut is a fundamental theorem: the maximum amount of flow from source to sink equals the minimum total capacity of edges that, if removed, would disconnect source from sink.

Q78. [Medium]

[BFS] — Rotting Oranges — Multi-source BFS

In the 'Rotting Oranges' problem, the answer (minimum minutes) equals:

- A) Number of fresh oranges
- ✓ **B) BFS levels from all initially rotten oranges minus 1 (or -1 if unreachable)**
- C) Total number of oranges
- D) Number of initially rotten oranges

Answer: B

Explanation: Multi-source BFS from all initially rotten oranges spreads in BFS levels. The number of levels traversed (minutes elapsed) until no fresh oranges remain is the answer.

Q79. [Hard]

[DSU] — Path Compression Variants

Which path compression technique achieves the best amortized complexity while being simpler to implement than full compression?

- A) Path splitting
- B) Path halving
- ✓ **C) Both path splitting and path halving achieve the same $O(\alpha(n))$ complexity**
- D) Rank compression only

Answer: C

Explanation: Both path splitting (make every node point to its grandparent on the find path) and path halving (same but every other node) achieve the same $O(\alpha(n))$ amortized complexity as full path compression, with simpler implementation.

Q80. [Hard]

[Graphs] — Johnson's Algorithm Use Case

Johnson's algorithm for all-pairs shortest paths is efficient when:

- A) Graph is dense
- ✓ **B) Graph is sparse — it runs in $O(V^2 \log V + VE)$, beating Floyd-Warshall's $O(V^3)$ for sparse graphs**
- C) All edge weights are positive
- D) Graph is a DAG

Answer: B

Explanation: Johnson's reweights edges using Bellman-Ford to eliminate negatives, then runs Dijkstra from each vertex. For sparse $E = O(V)$, this is $O(V^2 \log V)$, far better than Floyd-Warshall's $O(V^3)$.

Q81. [Medium]

[Tarjan] — SCC in Social Network Analysis

In a directed social graph (follow relationships), a Strongly Connected Component represents:

- A) Users with the most followers
- B) Users connected by exactly two edges
- ✓ **C) A group where every user can reach every other user by following links**
- D) A bipartite subgraph

Answer: C

Explanation: An SCC is a maximal subgraph where every vertex is reachable from every other. In social networks, this represents closed communities where all members can communicate through directed follow chains.

Q82. [Hard]

[Graphs] — Biconnected Components vs SCCs

Biconnected components apply to which type of graph?

- ✓ A) Undirected graphs — maximal subgraphs with no articulation points
- B) Directed graphs only
- C) Weighted graphs
- D) DAGs only

Answer: A

Explanation: Biconnected components partition the edges of undirected graphs into maximal subgraphs that remain connected when any single vertex is removed. SCCs apply to directed graphs.

Q83. [Hard]

[BFS] — BFS on Implicit Graph — Sliding Puzzle

For the 8-puzzle (3x3 sliding tiles), BFS finds the minimum moves. The state space size is:

- A) 8!
- ✓ B) $9!/2 = 181,440$ reachable states
- C) 9^9
- D) 2^9

Answer: B

Explanation: The 9 tiles can be arranged in 9! ways, but exactly half are reachable from any given state (parity invariant), giving $9!/2 = 181,440$ reachable states. BFS is feasible.

Q84. [Easy]

[DFS] — DFS on Grid — Number of Islands

DFS-based 'Number of Islands' solution: what terminates the recursion for each island?

- A) Visiting all grid cells
- ✓ B) Hitting boundary or water cells ('0') or already-visited cells
- C) Finding exactly 4 neighbors
- D) Reaching grid center

Answer: B

Explanation: DFS from a '1' cell recursively marks connected land cells as visited (e.g., '0' or '#'). Recursion stops at boundaries, water cells, or already-visited cells. Each DFS call from an unvisited '1' finds one island.

Q85. [Hard]

[Graphs] — Strongly Connected Component — Kosaraju's Two-Pass

Kosaraju's SCC algorithm performs:

- A) BFS twice on the original graph
- ✓ B) DFS on original graph (recording finish order), then DFS on reversed graph in reverse finish order
- C) DFS once with back-edge tracking
- D) BFS + topological sort

Answer: B

Explanation: Pass 1: DFS on G, push vertices to stack in finish order. Pass 2: reverse G, pop vertices and DFS; each DFS tree in pass 2 is an SCC. Two full DFS passes, $O(V+E)$ total.

Q86. [Medium]

[Graphs] — Minimum Spanning Tree Uniqueness

An MST is unique when:

- A) The graph is directed
- B) All edge weights are integers
- ✓ C) All edge weights are distinct
- D) The graph is bipartite

Answer: C

Explanation: If all edge weights are distinct, the MST is unique. The proof uses the cycle property: the unique maximum-weight edge in any cycle is never in the MST. With ties, multiple MSTs of the same weight may exist.

Q87. [Hard]

[BFS] — 0-1 BFS with Deque

0-1 BFS solves shortest path where edges have weight 0 or 1 using a deque. Why is it $O(V+E)$ instead of $O((V+E) \log V)$?

- A) Uses a Fibonacci heap internally
- ✓ B) Weight-0 edges push to front of deque, weight-1 to back, maintaining sorted order without heap overhead
- C) Processes only weight-0 edges
- D) Skips already-visited vertices

Answer: B

Explanation: By pushing weight-0 edges to the front and weight-1 to the back, the deque always has the minimum-distance node at the front — like a heap but $O(1)$ per push, giving $O(V+E)$ total.

Q88. [Hard]

[Graphs] — Traveling Salesman — DP Bitmask

The bitmask DP solution for TSP on n cities has complexity:

- A) $O(n^2)$
- B) $O(n!)$
- ✓ C) $O(2^n * n^2)$
- D) $O(n^3)$

Answer: C

*Explanation: State: $dp[mask][i]$ = min cost to visit cities in 'mask', ending at city i . There are $O(2^n * n)$ states, each requiring $O(n)$ transitions. Total: $O(2^n * n^2)$. Still exponential but much better than $O(n!)$.*

Q89. [Hard]

[DSU] — DSU with Rollback for Offline Queries

DSU with rollback (union by rank without path compression) supports undo in $O(\log n)$. Why can't path compression be used with rollback?

- A) Path compression is slower
- B) Path compression uses more memory
- ✓ C) Path compression irreversibly modifies parent pointers, making undo impossible without storing entire path history
- D) Path compression conflicts with rank

Answer: C

Explanation: Path compression restructures the tree, flattening it. To undo a union, you need to restore the original parent pointers. With path compression, intermediate pointers were changed and aren't easy to restore. Union by rank only changes two pointers per union, trivially undoable.

Q90. [Hard]

[Graphs] — Network Flow — Dinic's Algorithm Complexity

Dinic's algorithm for max flow has complexity:

- A) $O(V * E)$
- B) $O(E^2 * \log V)$
- ✓ C) $O(V^2 * E)$
- D) $O(V * E^2)$

Answer: C

*Explanation: Dinic's builds blocking flows in a level graph using BFS + DFS. There are $O(V)$ phases (BFS levels increase by at least 1 each phase), each blocking flow takes $O(VE)$. Total: $O(V^2 * E)$, or $O(E * \sqrt{V})$ for unit capacity networks.*

Q91. [Hard]

[Graphs] — Shortest Cycle Detection

Finding the shortest cycle (girth) in an unweighted undirected graph runs in:

- A) $O(V + E)$
- ✓ B) $O(V * (V + E))$ — BFS from each vertex
- C) $O(V^2)$
- D) $O(E \log V)$

Answer: B

Explanation: Run BFS from each vertex; when BFS finds an edge to an already-visited same-level node, a cycle is detected. The shortest such cycle across all BFS runs is the girth. V BFS runs of $O(V+E)$ each: $O(V(V+E))$.*

Q92. [Hard]

[DFS] — Boggle Solver — DFS + Trie

A Boggle solver DFS on an $N \times N$ board with a Trie of W words. What is the time complexity?

- A) $O(N^2 * W)$
- B) $O(N^2 * 4^{(N^2)})$
- ✓ C) $O(N^2 * 4^L)$ where L is max word length, pruned by Trie
- D) $O(W * N^2)$

Answer: C

*Explanation: DFS from each of N^2 cells explores at most 4^L paths of length L . The Trie prunes branches with no matching prefix early. In practice, much faster, but the theoretical bound is $O(N^2 * 4^L)$.*

Q93. [Hard]

[Graphs] — Minimum Cost Maximum Flow Application

Min-Cost Max-Flow is used to solve which type of problem?

- A) Minimum spanning tree
- B) Shortest path in DAG
- ✓ C) Assignment problems and transportation optimization (e.g., minimizing cost while satisfying supply-demand constraints)
- D) Topological sort

Answer: C

Explanation: Min-Cost Max-Flow finds the maximum flow from source to sink while minimizing total cost. It solves assignment problems, job scheduling, and transportation problems where both flow and cost must be optimized.

Q94. [Medium]

[BFS] — Alien Dictionary — Topological Sort

Given a sorted list of alien words, the character ordering is derived by:

- A) Comparing all pairs of characters
- ✓ B) Finding the first differing character between adjacent words to build edges, then topological sort
- C) Sorting characters by frequency
- D) DFS on character frequencies

Answer: B

Explanation: Compare each adjacent pair of words; the first character difference gives a directed edge ($c1 \rightarrow c2$ meaning $c1$ comes before $c2$ in the alien alphabet). Topological sort on these edges gives the ordering.

Q95. [Hard]

[Tarjan] — Critical Connections in Network — Bridge Finding

LeetCode 'Critical Connections in Network': which algorithm directly solves this?

- A) Dijkstra's
- B) BFS
- ✓ C) Tarjan's bridge-finding algorithm
- D) Floyd-Warshall

Answer: C

Explanation: Critical connections are edges whose removal increases the number of connected components — exactly the definition of bridges. Tarjan's algorithm finds all bridges in $O(V+E)$.

Q96. [Hard]

[Graphs] — Bipartite Matching — Hopcroft-Karp Complexity

Hopcroft-Karp algorithm for maximum bipartite matching runs in:

- A) $O(V * E)$
- ✓ B) $O(E * \sqrt{V})$
- C) $O(V^2)$
- D) $O(E \log V)$

Answer: B

*Explanation: Hopcroft-Karp finds maximal augmenting paths in phases using BFS + DFS. There are $O(\sqrt{V})$ phases, each requiring $O(E)$ work. Total: $O(E * \sqrt{V})$, significantly better than the $O(VE)$ naive approach.*

Q97. [Hard]

[DSU] — Weighted DSU for Relative Ranking

Weighted DSU (Union-Find with edge weights) can solve which type of problem?

- A) Finding MST directly
- ✓ B) Problems involving relative ordering or distance between elements (e.g., 'how many times heavier is A than B?')
- C) Detecting negative cycles
- D) Topological sort

Answer: B

Explanation: Weighted DSU maintains a weight relative to the root for each node. It can answer questions like 'what is the ratio between elements X and Y?' — used in problems like 'Evaluate Division' on LeetCode.

Q98. [Hard]

[Graphs] — PageRank — Graph Centrality

PageRank is computed iteratively as a form of:

- A) BFS from each vertex
- B) Topological sort of web pages
- ✓ C) Power iteration on the graph's adjacency matrix (dominant eigenvector computation)
- D) Dijkstra's on page link weights

Answer: C

Explanation: PageRank iteratively applies the transition matrix (normalized adjacency) until convergence, computing the stationary distribution of a random walk — equivalent to finding the dominant eigenvector of the transition matrix.

Q99. [Medium]

[BFS] — Jump Game II — BFS Greedy Levels

Jump Game II (minimum jumps to reach end) can be modeled as BFS where each 'level' represents:

- A) Each index in the array
- B) Each unique jump length
- ✓ C) All indices reachable with exactly k jumps (implicit BFS levels)
- D) Sorted jump distances

Answer: C

Explanation: Track the farthest index reachable within each 'level' (same number of jumps). When the current position crosses the previous level's boundary, increment jumps. This greedy BFS-level approach runs in $O(n)$.

Q100. [Easy]

[Graphs] — Course Schedule II — Prerequisites as DAG

'Course Schedule II' maps to finding a topological sort of a directed graph. If the graph has a cycle:

- A) Return an arbitrary ordering
- ✓ B) Return empty array — no valid course ordering exists
- C) Return the cycle nodes
- D) Skip cycled courses

Answer: B

Explanation: If prerequisites form a cycle (A requires B, B requires C, C requires A), no valid ordering exists.

Kahn's algorithm detects this when the topological sort output has fewer vertices than the total number of courses.

SECTION III — RECURSION & DYNAMIC PROGRAMMING

Q101. [Easy]

[Dynamic Programming] — 0/1 Knapsack State Definition

In 0/1 Knapsack with n items and capacity W , the DP state $dp[i][w]$ represents:

- A) Number of items selected up to index i
- B) Remaining capacity after i items
- ✓ C) Maximum value achievable using items $0..i$ with weight limit w
- D) Minimum weight for value i

Answer: C

Explanation: $dp[i][w]$ = max value using first i items with weight capacity w . Recurrence: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}[i]] + \text{value}[i])$ if $\text{weight}[i] \leq w$, else $dp[i-1][w]$.

Q102. [Medium]

[Dynamic Programming] — Knapsack Space Optimization

0/1 Knapsack can be solved in $O(W)$ space (instead of $O(n*W)$) by:

```
// Space-optimized 0/1 Knapsack
for (int i = 0; i < n; i++)
    for (int w = W; w >= weight[i]; w--) // iterate BACKWARDS
        dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);
```

- ✓ A) Iterating capacity from W down to $\text{weight}[i]$ prevents using item i twice
- B) Iterating from 0 to W
- C) Using two arrays and swapping
- D) Sorting items by value

Answer: A

Explanation: Backward iteration ensures that when we compute $dp[w]$, $dp[w - \text{weight}[i]]$ still reflects the state BEFORE item i was considered (previous row), maintaining the 0/1 constraint. Forward iteration would allow picking item i multiple times (unbounded knapsack).

Q103. [Medium]

[Dynamic Programming] — Unbounded Knapsack vs 0/1

The key difference between 0/1 and Unbounded Knapsack recurrences:

A) Unbounded uses 2D DP

✓ **B) Unbounded: $dp[w] = \max(dp[w], dp[w - \text{weight}[i]] + \text{value}[i])$ with FORWARD iteration; 0/1 uses backward**

C) Unbounded sorts by weight

D) 0/1 allows item reuse

Answer: B

Explanation: In unbounded knapsack, each item can be used unlimited times. Forward iteration allows item i to be selected again in the same pass ($dp[w - \text{weight}[i]]$ already reflects item i being chosen). 0/1 uses backward to prevent this.

Q104. [Easy]

[Dynamic Programming] — LCS Length — Output Prediction

Compute LCS length for $s1 = \text{'ABCBDB'}$ and $s2 = \text{'BDCAB'}$.

A) 3

✓ **B) 4**

C) 5

D) 2

Answer: B

Explanation: The LCS is 'BCAB' or 'BDAB' of length 4. The DP table confirms this via the standard $O(m \cdot n)$ recurrence.

Q105. [Hard]

[Dynamic Programming] — LCS to LIS Reduction

Longest Increasing Subsequence (LIS) can be reduced to LCS by computing LCS of the array with:

A) Its reversed copy

✓ **B) Its sorted (distinct) copy**

C) An array of all 1s

D) The prefix sums

Answer: B

Explanation: $LIS(A) = LCS(A, \text{sorted_unique}(A))$. An element appears in the LIS iff it appears in both A (in order) and the sorted unique version. This runs in $O(n^2)$ via LCS; patience sorting gives $O(n \log n)$ directly.

Q106. [Medium]

[Dynamic Programming] — Edit Distance — Three Operations

The edit distance (Levenshtein) DP recurrence $dp[i][j] = ?$

A) $dp[i-1][j-1] + 1$ always

✓ **B) If $s[i] == t[j]$: $dp[i-1][j-1]$; else $1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$**

C) $\min(dp[i-1][j], dp[i][j-1])$

D) $dp[i-1][j-1] + (s[i] != t[j])$

Answer: B

Explanation: If characters match, no operation needed: take $dp[i-1][j-1]$. Otherwise, take minimum of: delete from s ($dp[i-1][j] + 1$), insert into s ($dp[i][j-1] + 1$), or replace ($dp[i-1][j-1] + 1$).

Q107. [Easy]

[Dynamic Programming] — Matrix Chain Multiplication — State

In Matrix Chain Multiplication DP, $dp[i][j]$ represents:

- A) Cost to multiply matrix i and j
- ✓ **B) Minimum scalar multiplications to compute the product of matrices i through j**
- C) Dimensions of matrix ij
- D) Number of ways to parenthesize matrices $i..j$

Answer: B

Explanation: $dp[i][j]$ = min cost to multiply the chain of matrices from index i to j . Recurrence: $dp[i][j] = \min \text{ over } k \text{ of } (dp[i][k] + dp[k+1][j] + \text{dims}[i-1] * \text{dims}[k] * \text{dims}[j])$.

Q108. [Medium]

[Dynamic Programming] — MCM Time Complexity

Matrix Chain Multiplication DP has time complexity:

- A) $O(n^2)$
- ✓ **B) $O(n^3)$**
- C) $O(2^n)$
- D) $O(n^2 \log n)$

Answer: B

Explanation: There are $O(n^2)$ subproblems (all pairs i, j), and each requires $O(n)$ work to try all split points k . Total: $O(n^3)$. Space is $O(n^2)$.

Q109. [Easy]

[Recursion] — Tower of Hanoi — Minimum Moves

The minimum number of moves to solve Tower of Hanoi with n disks is:

- A) n^2
- ✓ **B) $2^n - 1$**
- C) $n!$
- D) $n*(n+1)/2$

Answer: B

Explanation: $T(n) = 2 * T(n-1) + 1$ (move top $n-1$ to auxiliary, move n th disk, move $n-1$ back). Solving: $T(n) = 2^n - 1$. For 64 disks: $2^{64} - 1 \approx 1.8 \times 10^{19}$ moves.

Q110. [Medium]

[Dynamic Programming] — Coin Change — Fewest Coins

For coin denominations [1,5,6,9] and amount 11, the greedy approach (pick largest) gives 3 coins (9+1+1). DP gives:

- A) 3 coins
- ✓ **B) 2 coins (5+6)**
- C) 4 coins
- D) 1 coin

Answer: B

Explanation: Greedy fails here. DP correctly identifies $5+6=11$ using 2 coins, proving that coin change requires DP (or BFS), not greedy, for arbitrary denominations.

Q111. [Medium]

[Dynamic Programming] — Longest Palindromic Subsequence

LPS(s) is related to LCS by:

- A) $LPS(s) = LCS(s, reversed(s)) - len(s)$
- ✓ B) $LPS(s) = LCS(s, reversed(s))$
- C) $LPS(s) = 2 * LCS(s, reversed(s))$
- D) $LPS(s) = len(s) - LCS(s, reversed(s))$

Answer: B

Explanation: A palindromic subsequence reads the same forwards and backwards, so it's a common subsequence of s and its reverse. $LPS(s) = LCS(s, reverse(s))$.

Q112. [Hard]

[Dynamic Programming] — Wildcard Matching — DP Bug

Identify if this wildcard matching DP has a bug:

```
bool isMatch(string s, string p) {
    int m=s.size(), n=p.size();
    vector<vector<bool>> dp(m+1, vector<bool>(n+1, false));
    dp[0][0] = true;
    for (int j=1; j<=n; j++)
        if (p[j-1]=='*') dp[0][j] = dp[0][j-1]; // handle leading *s
    for (int i=1; i<=m; i++)
        for (int j=1; j<=n; j++) {
            if (p[j-1]=='*')
                dp[i][j] = dp[i-1][j] || dp[i][j-1];
            else
                dp[i][j] = dp[i-1][j-1] && (s[i-1]==p[j-1] || p[j-1]=='?');
        }
    return dp[m][n];
}
```

✓ A) No bug — this is correct. $dp[i-1][j]$ handles '*' matching one+ chars, $dp[i][j-1]$ handles '*' matching empty

- B) '*' should only match $dp[i-1][j-1]$
- C) '?' should match any number of chars
- D) Missing check for $p[j-1] == '?'$

Answer: A

Explanation: The code is correct. '' can match any sequence: $dp[i-1][j]$ means '*' matches one more character from s; $dp[i][j-1]$ means '*' matches empty string. The initialization correctly handles patterns starting with '*'.*

Q113. [Medium]

[Dynamic Programming] — Partition Equal Subset Sum

'Partition Equal Subset Sum' reduces to which well-known DP problem?

- A) Coin Change
- B) LCS
- ✓ C) 0/1 Knapsack with target = total_sum / 2
- D) Edit Distance

Answer: C

Explanation: If total sum is odd, return false. Otherwise, check if a subset sums to total/2 using 0/1 Knapsack DP. $dp[w] = \text{true}$ if some subset sums to exactly w .

Q114. [Medium]

[Recursion] — Memoization vs Tabulation Trade-offs

When is top-down memoization preferred over bottom-up tabulation?

- A) Always — memoization is faster
- ✓ B) When only a fraction of subproblems need to be computed (sparse dependency graph)
- C) When space is the primary constraint
- D) For iterative problems

Answer: B

Explanation: If the recursion only visits a small fraction of all possible states (e.g., a sparse dependency graph), memoization avoids computing unnecessary states. Tabulation always fills all states, even irrelevant ones.

Q115. [Medium]

[Dynamic Programming] — Rod Cutting — Unbounded Knapsack Variant

Rod cutting (maximize revenue from cutting a rod of length n with given piece prices) is a variant of:

- A) 0/1 Knapsack
- ✓ B) Unbounded Knapsack — each piece length can be used multiple times
- C) Fractional Knapsack
- D) Matrix Chain Multiplication

Answer: B

Explanation: Each cut length can be reused any number of times (a piece of length 2 can be cut multiple times from a rod of length 8). This maps directly to Unbounded Knapsack.

Q116. [Hard]

[Dynamic Programming] — Burst Balloons — Interval DP

'Burst Balloons' uses interval DP where $dp[i][j]$ means:

- A) Number of ways to burst balloons i to j
- ✓ B) Maximum coins from bursting all balloons between i and j (exclusive boundaries)
- C) Minimum coins to burst balloons i to j
- D) Sum of balloon values from i to j

Answer: B

Explanation: $dp[i][j]$ = max coins obtainable by bursting all balloons strictly between indices i and j (i and j are boundaries not burst in this range). The key insight: think of k as the LAST balloon burst in range (i,j) , not the first.

Q117. [Easy]

[Dynamic Programming] — Longest Common Substring vs Subsequence

The key difference between LCS (Longest Common Subsequence) and Longest Common Substring (LCStr):

A) LCS is $O(n^2)$, LCStr is $O(n^3)$

✓ **B) LCS allows gaps (subsequence); LCStr requires contiguity. LCStr resets $dp[i][j]$ to 0 when characters don't match**

C) LCStr is always longer than LCS

D) They use identical recurrences

Answer: B

Explanation: LCS: $dp[i][j] = dp[i-1][j-1] + 1$ if match, else $\max(dp[i-1][j], dp[i][j-1])$. LCStr: $dp[i][j] = dp[i-1][j-1] + 1$ if match, else 0. LCStr tracks the max seen; no gap allowed.

Q118. [Hard]

[Dynamic Programming] — DP on Trees — Maximum Independent Set

Maximum Independent Set on a tree: $dp[v][0]$ = max nodes in subtree of v when v is NOT selected, $dp[v][1]$ = max when v IS selected. Recurrence for $dp[v][1]$?

A) $1 + \text{sum of } dp[c][1] \text{ for children } c$

✓ **B) $1 + \text{sum of } dp[c][0] \text{ for children } c$ — if v is selected, no child can be selected**

C) $\text{sum of } \max(dp[c][0], dp[c][1]) \text{ for children } c$

D) $dp[\text{parent}][0] + 1$

Answer: B

Explanation: If vertex v is included, none of its children can be included (independent set constraint). So $dp[v][1] = 1 + \text{sum}(dp[c][0])$ over all children c . $dp[v][0] = \text{sum}(\max(dp[c][0], dp[c][1]))$ since children can independently be in or out.

Q119. [Hard]

[Dynamic Programming] — Palindrome Partitioning — DP State

'Palindrome Partitioning II' (minimum cuts for palindromic partition): $dp[i]$ represents:

A) Number of palindromes ending at i

✓ **B) Minimum cuts needed for $s[0..i]$**

C) Length of longest palindrome ending at i

D) Number of ways to partition $s[0..i]$

Answer: B

Explanation: $dp[i] = \text{min cuts for } s[0..i]$. Transition: $dp[i] = \min(dp[j-1] + 1)$ for all $j \leq i$ where $s[j..i]$ is a palindrome. Pre-compute palindrome table using DP or Manacher's $O(n)$ algorithm for optimization.

Q120. [Easy]

[Recursion] — Subset Sum — Exponential Baseline

Naive recursive subset sum explores how many subsets for n elements?

A) n^2

B) $n!$

✓ **C) 2^n**

D) n^3

Answer: C

Explanation: Each of n elements can be included or excluded: 2^n subsets. Naive recursion without memoization explores all 2^n possibilities. DP reduces this to $O(n \cdot W)$ pseudo-polynomial time.

Q121. [Hard]

[Dynamic Programming] — Number of Distinct Subsequences

$dp[i][j]$ in 'Distinct Subsequences' (count ways s 's subsequences equal t) represents:

- A) Length of LCS up to $s[i]$ and $t[j]$
- ✓ **B) Number of ways $s[0..i-1]$ contains $t[0..j-1]$ as a subsequence**
- C) Edit distance between $s[0..i]$ and $t[0..j]$
- D) Number of common characters

Answer: B

Explanation: $dp[i][j] = dp[i-1][j] + dp[i-1][j-1]$ if $s[i-1] == t[j-1]$ (use $s[i]$ to match $t[j]$). Base: $dp[i][0] = 1$ (empty t matched always), $dp[0][j] = 0$ for $j > 0$.

Q122. [Hard]

[Dynamic Programming] — Largest Rectangle in Histogram — Stack DP

The 'Largest Rectangle in Histogram' optimal solution uses:

- A) $O(n^2)$ DP
- B) Segment tree for range minimum
- ✓ **C) Monotonic stack in $O(n)$ time**
- D) Divide and conquer in $O(n \log n)$

Answer: C

Explanation: A monotonic increasing stack tracks bars. When a shorter bar is encountered, pop bars and compute rectangle area (popped height * width since last shorter bar). $O(n)$ total since each bar is pushed/popped once.

Q123. [Medium]

[Dynamic Programming] — Catalan Numbers — DP

Catalan number C_n counts many things. $C_n = \sum C_i * C_{n-1-i}$ for $i=0$ to $n-1$. This recurrence relates to:

- A) Number of spanning trees of K_n
- ✓ **B) Number of valid BSTs with n keys, number of valid parenthesizations, Dyck paths**
- C) Number of prime numbers up to n
- D) Number of permutations of n elements

Answer: B

Explanation: Catalan numbers count: valid BSTs with n distinct keys, ways to parenthesize $n+1$ matrices (MCM), Dyck paths of length $2n$, valid bracket sequences of length $2n$, triangulations of polygon with $n+2$ sides, and many more.

Q124. [Medium]

[Recursion] — Generate Permutations — Backtracking Complexity

Generating all permutations of n distinct elements via backtracking has time complexity:

- A) $O(n^2)$
- ✓ **B) $O(n * n!)$**
- C) $O(n!)$
- D) $O(2^n)$

Answer: B

Explanation: There are $n!$ permutations, each of length n to write/copy. Total work: $O(n * n!)$. The recursion tree has $n!$ leaves, but internal nodes sum to another factor.

Q125. [Hard]

[Dynamic Programming] — Stock Buy Sell with Cooldown

'Best Time to Buy and Sell Stock with Cooldown': after selling, you must wait 1 day. The DP states needed are:

- A) $dp[i]$ = max profit on day i
- ✓ **B) $held[i]$, $sold[i]$, $rest[i]$ — three states per day**
- C) $buy[i]$, $sell[i]$ — two states
- D) $dp[i][j]$ where j is number of transactions

Answer: B

Explanation: Three states capture all transitions: $held[i]$ = max profit holding stock on day i ; $sold[i]$ = max profit just sold on day i ; $rest[i]$ = max profit in cooldown/rest on day i . Transitions encode the cooldown rule naturally.

Q126. [Hard]

[Dynamic Programming] — Interleaving String — 2D DP

'Interleaving String' (can $s3$ be formed by interleaving $s1$ and $s2$) uses $dp[i][j]$ where:

- A) $dp[i][j]$ = LCS of $s1[0..i]$ and $s2[0..j]$
- ✓ **B) $dp[i][j]$ = true if $s3[0..i+j-1]$ can be formed by interleaving $s1[0..i-1]$ and $s2[0..j-1]$**
- C) $dp[i][j]$ = edit distance between $s1[0..i]$ and $s3[0..j]$
- D) $dp[i][j]$ = number of interleavings

Answer: B

Explanation: $dp[i][j] = (dp[i-1][j] \ \&\& \ s1[i-1] == s3[i+j-1]) \ || \ (dp[i][j-1] \ \&\& \ s2[j-1] == s3[i+j-1])$. Base case $dp[0][0] = true$. Answer is $dp[m][n]$.

Q127. [Medium]

[Dynamic Programming] — Palindrome Substrings — Count

'Count Palindromic Substrings': the expand-around-center technique runs in:

- A) $O(n^3)$
- ✓ **B) $O(n^2)$**
- C) $O(n)$ with Manacher's (but Manacher's finds longest, not count directly)
- D) $O(n \log n)$

Answer: B

Explanation: For each of the $2n-1$ centers (n odd-length, $n-1$ even-length), expand while palindrome. Total expansions across all centers: $O(n^2)$. Manacher's achieves $O(n)$ for the longest palindrome but counting all requires $O(n^2)$ without extensions.

Q128. [Easy]

[Dynamic Programming] — Jump Game II — DP vs Greedy

Jump Game II solved with DP has $O(n^2)$ complexity. The greedy approach achieves:

- A) $O(n^2)$
- ✓ **B) $O(n)$**
- C) $O(n \log n)$
- D) $O(1)$

Answer: B

Explanation: Greedy: track current range end and farthest reachable. When index reaches range end, increment jumps and extend range to farthest. Single $O(n)$ pass suffices, avoiding the $O(n^2)$ DP.

Q129. [Medium]

[Recursion] — N-Queens — Backtracking State

N-Queens backtracking maintains which sets for $O(1)$ conflict checking?

- A) Rows only
- ✓ **B) Sets for: columns, left diagonals (row-col), and right diagonals (row+col)**
- C) Sets for rows and columns
- D) A 2D boolean board only

Answer: B

Explanation: Three hash sets: columns used, left diagonals (constant row-col), right diagonals (constant row+col). Checking if a position (r,c) is safe is $O(1)$ with these sets vs $O(n)$ scanning the board.

Q130. [Hard]

[Dynamic Programming] — Longest Increasing Path in Matrix

'Longest Increasing Path in a Matrix' uses memoized DFS. The time complexity is:

- A) $O(m*n * 4^{(m*n)})$
- ✓ **B) $O(m*n)$ — each cell computed once via memoization**
- C) $O((m*n)^2)$
- D) $O(m*n * \log(m*n))$

Answer: B

*Explanation: Each cell is the starting point of at most one DFS computation (subsequent calls hit the memo cache). Each cell has at most 4 neighbors. Total work: $O(4*m*n) = O(m*n)$.*

Q131. [Medium]

[Dynamic Programming] — Decode Ways — State Machine DP

'Decode Ways' (count ways to decode a digit string as letters): $dp[i]$ depends on:

- A) $dp[i-1]$ only
- B) $dp[i-2]$ only
- ✓ **C) $dp[i-1]$ if $s[i]$ is valid single digit, + $dp[i-2]$ if $s[i-1..i]$ is valid two-digit (10-26)**
- D) All previous states

Answer: C

Explanation: At each position i , you can decode one digit (if valid, adds $dp[i-1]$ ways) or two digits (if 10-26, adds $dp[i-2]$ ways). Handle '0' carefully: '0' alone is invalid; only '10' and '20' are valid two-digit.

Q132. [Medium]

[Dynamic Programming] — Word Break — DP + Trie

'Word Break' (can string s be segmented into dictionary words) with a Trie for the dictionary runs in:

- A) $O(n^2)$
- ✓ **B) $O(n^2)$ with hash set, or $O(n * L)$ with Trie where L is max word length**
- C) $O(n * W)$ where W is total characters in dictionary
- D) $O(n^3)$

Answer: B

*Explanation: $dp[i] = \text{true}$ if $s[0..i-1]$ is segmentable. For each i , check all $j < i$: $dp[j] \ \&\& \ s[j..i-1]$ in dictionary. With hash set: $O(n^2)$ average. With Trie: at most L characters per position, so $O(n*L)$.*

Q133. [Easy]**[Dynamic Programming] — Distinct BSTs — Catalan Number**

The number of structurally unique BSTs with n keys is the n -th Catalan number. For $n=4$:

- A) 12
- ✓ B) 14
- C) 16
- D) 10

Answer: B

*Explanation: Catalan numbers: $C(0)=1$, $C(1)=1$, $C(2)=2$, $C(3)=5$, $C(4)=14$. For each root k (1 to n), left subtree has $k-1$ keys, right has $n-k$. $C(n) = \sum(C(k)*C(n-1-k))$.*

Q134. [Medium]**[Recursion] — Memoization Space Overhead in Deep Recursion**

A top-down DP with n states, each requiring $O(k)$ space for its result, uses total space of:

- A) $O(n)$ for hash map
- ✓ B) $O(n*k)$ for results + $O(\text{depth})$ for call stack
- C) $O(n*\text{depth})$
- D) $O(k)$ only

Answer: B

*Explanation: The memo table stores n results, each of size k : $O(n*k)$. The recursion call stack uses $O(\text{depth}) = O(n)$ in the worst case (linear chain). Total space: $O(n*k + n) = O(n*k)$.*

Q135. [Easy]**[Dynamic Programming] — Maximum Subarray — Kadane's Insight**

Kadane's algorithm DP insight: $dp[i] = \text{max subarray sum ending at index } i =$

- A) $dp[i-1] + a[i]$
- ✓ B) $\max(a[i], dp[i-1] + a[i])$
- C) $\max(dp[i-1], a[i])$
- D) $dp[i-1] - a[i]$

Answer: B

Explanation: Either extend the previous subarray ($dp[i-1] + a[i]$) or start fresh at current element ($a[i]$). We choose whichever is larger. This compresses a 2D DP to 1D, running in $O(n)$ time and $O(1)$ space.

Q136. [Easy]**[Dynamic Programming] — Minimum Path Sum in Grid**

Grid DP for minimum path sum: $dp[i][j] = ?$

- A) $grid[i][j] + \min(dp[i][j-1], dp[i-1][j])$ always
- ✓ B) $grid[i][j] + \min(dp[i][j-1], dp[i-1][j])$, with boundary handling for first row/col
- C) $grid[i][j] * \min(dp[i][j-1], dp[i-1][j])$
- D) $\max(dp[i][j-1], dp[i-1][j]) - grid[i][j]$

Answer: B

Explanation: General recurrence: $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$. First row: only from left. First column: only from above. Answer: $dp[m-1][n-1]$.

Q137. [Medium]

[Dynamic Programming] — Dice Roll Target Sum — Count Ways

Counting ways to get sum S with d dice, each having f faces. The DP table has dimensions:

- A) $O(S)$
- B) $O(d * f)$
- ✓ C) $O(d * S)$
- D) $O(f * S)$

Answer: C

*Explanation: $dp[i][j]$ = number of ways to get sum j using i dice. i ranges from 1 to d ; j ranges from i to $i*f$. Table size: $O(d*S)$. Transition: $dp[i][j] = \sum(dp[i-1][j-k] \text{ for } k=1 \text{ to } f)$.*

Q138. [Medium]

[Recursion] — Combination Sum — Backtracking Termination

In Combination Sum backtracking (elements can be reused, find all combinations summing to target), what prevents infinite loops?

- A) Sorting candidates
- B) Visited array
- ✓ C) Remaining target decreases each recursive call; terminate when target == 0 or < 0
- D) Unique elements constraint

Answer: C

Explanation: Since all candidates are positive, subtracting any candidate strictly decreases the remaining target. Recursion terminates when target hits 0 (found combination) or goes negative (dead end). No risk of infinite recursion.

Q139. [Hard]

[Dynamic Programming] — Regular Expression Matching — DP

In regex matching DP with '.' (any char) and '*' (zero or more), $dp[i][j]$ when $p[j-1] == '*'$ equals:

- A) $dp[i-1][j-1]$
- B) $dp[i][j-2]$ only (zero occurrences)
- ✓ C) $dp[i][j-2]$ (zero occurrences) OR ($dp[i-1][j]$ if $s[i-1]$ matches $p[j-2]$)
- D) $dp[i-1][j]$

Answer: C

Explanation: '' can mean zero occurrences: $dp[i][j-2]$. Or one-or-more: $dp[i-1][j]$ if the preceding character $p[j-2]$ matches $s[i-1]$ (or $p[j-2] == '.'$). This is the critical case that handles multiple character repetitions.*

Q140. [Hard]

[Dynamic Programming] — Egg Drop — Optimal DP

The classic Egg Drop problem with k eggs and n floors: the naive $O(kn^2)$ DP can be optimized to $O(kn \log n)$ using:

- A) Greedy selection
- ✓ B) Binary search on the optimal floor to drop from (monotonic optimal structure)
- C) Divide and conquer
- D) Memoization only

Answer: B

Explanation: The 'check floor' function in the DP transition is monotonic: as the answer increases, the optimal drop floor only increases. Binary search exploits this property to find the optimal floor in $O(\log n)$ instead of $O(n)$, improving total complexity.

Q141. [Hard]

[Dynamic Programming] — Maximum Profit with K Transactions

'Best Time to Buy and Sell Stock IV' (at most k transactions) DP complexity:

- A) $O(n * k)$
- ✓ B) $O(n * k)$ time with $O(k)$ space optimization
- C) $O(n^2)$
- D) $O(n^2 * k)$

Answer: B

*Explanation: $dp[i][j]$ = max profit with at most j transactions up to day i . With space optimization (rolling array on transactions), $O(n*k)$ time and $O(k)$ space. If $k \geq n/2$, unlimited transactions (greedy) gives $O(n)$.*

Q142. [Easy]

[Recursion] — Subset Generation — Bitmask vs Backtracking

For generating all subsets of n elements, bitmask iteration vs backtracking have what time/space complexity?

- ✓ A) Both $O(2^n)$ time; bitmask $O(1)$ extra space, backtracking $O(n)$ stack space
- B) Bitmask $O(n*2^n)$, backtracking $O(2^n)$
- C) Bitmask $O(1)$, backtracking $O(2^n)$
- D) Both $O(n*2^n)$ time and $O(n)$ space

Answer: A

*Explanation: Both explore all 2^n subsets. Bitmask uses a loop variable ($O(1)$ space). Backtracking uses the recursion stack ($O(n)$ depth). To print each subset of size up to n , each subset takes $O(n)$, giving $O(n*2^n)$ output time for both.*

Q143. [Hard]

[Dynamic Programming] — Minimum Window Subsequence

'Minimum Window Subsequence' (find shortest substring of S containing T as subsequence) optimal approach is:

- A) Sliding window $O(n*m)$
- B) BFS on characters
- ✓ C) DP with backward scan: after finding T in S forward, scan backward to tighten window — $O(n*m)$ but optimal in practice
- D) Suffix array

Answer: C

*Explanation: Forward scan finds an end position where T is a subsequence. Backward scan from that end finds the tightest start. DP precomputes next occurrence of each character for $O(n)$ scanning. Total $O(n*m)$ or $O(n * |\text{alphabet}|)$ with preprocessing.*

Q144. [Medium]

[Dynamic Programming] — Maximal Square of 1s in Matrix

$dp[i][j]$ in 'Maximal Square' (largest square of 1s) represents:

- A) Number of 1s in row i up to column j
- ✓ B) Side length of the largest square with (i,j) as its bottom-right corner
- C) Sum of matrix[0..i][0..j]
- D) Number of squares ending at (i,j)

Answer: B

Explanation: $dp[i][j] = \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$ if $matrix[i][j] == '1'$, else 0. The minimum of the three neighboring DP values limits the largest square that can extend to (i,j). Answer: $\max(dp[i][j])^2$.

Q145. [Easy]

[Recursion] — Flood Fill — DFS vs BFS

Flood Fill (replace connected region of same color) can use DFS or BFS. For a worst-case $m \times n$ grid, stack overflow risk exists with DFS when:

- A) $m*n > 100$
- B) Grid is square
- ✓ C) $m*n$ is very large (millions of cells), causing recursion stack depth to exceed system limits
- D) When colors are repeated

Answer: C

*Explanation: In worst case, DFS recurses through all $m*n$ connected cells, creating a call stack of depth $O(m*n)$. For large grids, this exceeds typical stack limits ($\sim 10^4$ - 10^5 frames). BFS uses an explicit queue (heap-allocated) and avoids this.*

Q146. [Easy]**[Dynamic Programming] — Longest Common Subsequence — Space Optimization**

LCS(s_1 , s_2) of length m and n uses $O(m \cdot n)$ space. The space-optimized version uses:

- A) $O(n)$ using a single array
- ✓ **B) $O(n)$ using two rows (current and previous) or one array with careful traversal**
- C) $O(\min(m, n))$
- D) $O(1)$

Answer: B

Explanation: Since $dp[i][j]$ only depends on $dp[i-1][j]$, $dp[i][j-1]$, and $dp[i-1][j-1]$, we can use two 1D arrays (prev and curr) of size $n+1$, reducing space from $O(m \cdot n)$ to $O(n)$. With a single array and a temp variable, $O(n)$ space.

Q147. [Hard]**[Dynamic Programming] — Dice Game — Expected Value DP**

In a dice game where you roll a fair 6-sided die and can stop at any time to collect the rolled value (or reroll), what is the expected value of an optimal strategy?

- A) 3.5
- ✓ **B) Higher than 3.5 — reroll when value $\leq$ expected threshold, approximately 4.0**
- C) 2.5
- D) 6.0

Answer: B

Explanation: Let E be the expected value. You keep the roll if it's $> E$, otherwise reroll. $E = (1/6) \cdot \text{sum of values} > E + (P(\text{roll} \leq E)) \cdot E$. Solving: $E = 14/4 = 3.5$? With optimal play: keep if die shows 4, 5, 6; $E = (4+5+6)/3 \cdot (3/6) + E \cdot (3/6) \Rightarrow E/2 = 5 \Rightarrow E \approx 4.0$.

Q148. [Hard]**[Dynamic Programming] — Minimum Number of Refueling Stops**

'Minimum Refueling Stops' can be solved greedily using a max-heap. The greedy insight is:

- A) Always refuel at the first available station
- B) Always skip stations
- ✓ **C) When stuck (can't reach next station), retroactively pick the largest refuel amount seen so far from the heap**
- D) Sort stations by fuel amount

Answer: C

Explanation: Use a max-heap of fuel amounts at passed stations. When current fuel can't reach the next station, greedily pop the largest fuel from the heap (retroactively refueling there). Increment stops count. This greedy with heap is $O(n \log n)$.

Q149. [Easy]

[Recursion] — Generate Valid Parentheses — Counting

The number of valid parentheses strings with n pairs equals:

- A) 2^n
- B) $n!$
- ✓ C) $C(n)$ — the n -th Catalan number
- D) n^2

Answer: C

Explanation: Valid parentheses strings with n pairs are counted by Catalan numbers: $C(0)=1$, $C(1)=1$, $C(2)=2$, $C(3)=5$, $C(4)=14$. The backtracking generates all $C(n)$ solutions.

Q150. [Hard]

[Dynamic Programming] — Buying and Selling Stock — State Machine Summary

Which state machine correctly models 'Best Time to Buy and Sell Stock III' (at most 2 transactions)?

```
// States: buy1, sell1, buy2, sell2
buy1 = max(buy1, -price)
sell1 = max(sell1, buy1 + price)
buy2 = max(buy2, sell1 - price)
sell2 = max(sell2, buy2 + price)
```

- A) Incorrect — should use 2D DP only
- ✓ B) Correct — represents optimal profit at each stage of two transactions
- C) Incorrect — buy2 should not depend on sell1
- D) Correct only for unlimited transactions

Answer: B

Explanation: This elegant state machine captures two transaction stages. buy1: best profit if currently holding stock from first buy. sell1: best profit after first sale. buy2: best profit after buying second stock (uses sell1 profit). sell2: final profit after second sale.

SECTION IV — LANGUAGE INTERNALS & CODE COMPREHENSION

Q151. [Easy]

[C++ Internals] — std::vector Capacity Doubling

After the following operations, what is the capacity of v?

```
std::vector<int> v;  
for (int i = 0; i < 17; i++)  
    v.push_back(i);  
// Assuming initial capacity = 1, doubling strategy
```

- A) 17
- B) 18
- ✓ C) 32
- D) 16

Answer: C

Explanation: Starting at capacity 1: grows to 2→4→8→16 (after 16 pushes). The 17th push triggers another doubling to 32. GCC/Clang typically use doubling; MSVC uses 1.5x growth. The capacity is 32.

Q152. [Easy]

[C++ Internals] — std::vector vs std::deque Access Complexity

What is the complexity of random access by index for std::vector vs std::deque?

- ✓ A) vector: O(1); deque: O(1) but with larger constant
- B) vector: O(1); deque: O(n)
- C) Both O(log n)
- D) vector: O(n); deque: O(1)

Answer: A

Explanation: std::vector stores contiguous memory: O(1) by pointer arithmetic. std::deque uses chunked storage (array of pointers to fixed-size blocks): O(1) mathematically (two-level index) but with larger constant due to indirection.

Q153. [Medium]

[C++ Internals] — Dangling Iterator After Reallocation

What happens to an iterator to a std::vector element after push_back causes reallocation?

```
vector<int> v = {1, 2, 3};  
auto it = v.begin();  
v.push_back(4); // May trigger reallocation  
cout << *it; // ?
```

- A) Prints 1 always
- ✓ B) Undefined behavior — iterator is invalidated if reallocation occurs
- C) Prints 0
- D) Throws std::bad_iterator

Answer: B

Explanation: If push_back triggers reallocation, all iterators, pointers, and references to vector elements are invalidated. Dereferencing them is undefined behavior. Use index-based access to avoid this.

Q154. [Medium]

[C++ Internals] — std::unordered_map Worst-Case Lookup

In the worst case (all keys hash to same bucket), std::unordered_map lookup is:

✓ **A) $O(n)$ — degrades to linked list traversal**

B) $O(\log n)$

C) $O(1)$ always

D) $O(n \log n)$

Answer: A

Explanation: std::unordered_map uses separate chaining. With all n keys in one bucket, lookup traverses the entire chain: $O(n)$. This occurs with hash collision attacks. Since C++11, some implementations switch to RB-tree ($O(\log n)$) when a bucket exceeds a threshold.

Q155. [Easy]

[C++ Internals] — Smart Pointer — shared_ptr Reference Count

What does this code print?

```
#include <memory>
#include <iostream>
int main() {
    auto p1 = std::make_shared<int>(42);
    auto p2 = p1;
    auto p3 = p1;
    std::cout << p1.use_count() << std::endl;
    p2.reset();
    std::cout << p1.use_count() << std::endl;
}
```

A) 2 then 1

✓ **B) 3 then 2**

C) 1 then 0

D) 3 then 1

Answer: B

Explanation: After make_shared and two copies, use_count=3. After p2.reset(), one reference dropped: use_count=2. p1 and p3 still share ownership. Memory freed only when count reaches 0.

Q156. [Medium]

[C++ Internals] — Move Semantics — std::move Effect

After std::move, what is the state of the moved-from object?

```
std::string s1 = "hello";
std::string s2 = std::move(s1);
std::cout << s1.size();
```

- A) 5 — s1 unchanged
- ✓ B) 0 or unspecified — s1 is in a valid but unspecified state
- C) Undefined behavior
- D) Compilation error

Answer: B

Explanation: After std::move, s1 is in a valid but unspecified state (standard guarantee). In practice, s1.size() returns 0 (moved-from string is empty), but you should not rely on specific state. Accessing size() is safe; using content is not.

Q157. [Easy]

[C++ Internals] — RAII and Exception Safety

Why does using std::unique_ptr instead of raw new/delete guarantee no memory leak even with exceptions?

- A) unique_ptr catches exceptions
- B) unique_ptr uses try-catch internally
- ✓ C) unique_ptr destructor runs during stack unwinding, freeing memory regardless of how scope is exited
- D) Exceptions don't affect unique_ptr

Answer: C

Explanation: RAII: resource acquisition in constructor, release in destructor. When an exception unwinds the stack, destructors of all local objects (including unique_ptr) are called in reverse order of construction, ensuring cleanup regardless of exception.

Q158. [Medium]

[C++ Internals] — Virtual Destructor Necessity

Why must a base class destructor be virtual if derived objects are deleted via base pointer?

```
class Base { public: ~Base() { /* no virtual */ } };
class Derived : public Base { int* data; public:
Derived() { data = new int[100]; }
~Derived() { delete[] data; } // never called?
};
Base* b = new Derived();
delete b; // Problem?
```

- A) Compiles but runs correctly
- ✓ B) Undefined behavior — ~Derived() not called, causes memory leak
- C) Throws exception at runtime
- D) delete calls both destructors automatically

Answer: B

Explanation: Without virtual destructor, `delete b` calls Base::~~Base() (static dispatch on Base). Derived::~~Derived() is never called, so `data` array is never freed — memory leak and UB. Always declare base class destructor virtual.*

Q159. [Easy]

[C Internals] — malloc vs calloc

What is the key behavioral difference between malloc and calloc?

- A) malloc is faster always
- ✓ **B) malloc does not initialize memory; calloc zero-initializes the allocated memory**
- C) calloc takes one argument; malloc takes two
- D) malloc is for single objects; calloc is for arrays

Answer: B

*Explanation: malloc(n) allocates n bytes, content undefined. calloc(n, size) allocates n*size bytes AND zero-initializes them. calloc(3,4) is conceptually equivalent to memset(malloc(12), 0, 12) but calloc may be OS-optimized (requesting zeroed pages).*

Q160. [Medium]

[C Internals] — realloc Behavior on Failure

What is the bug in this realloc usage?

```
int* arr = (int*)malloc(10 * sizeof(int));
arr = (int*)realloc(arr, 20 * sizeof(int)); // BUG?
if (!arr) { /* handle error */ }
```

- A) realloc should not be cast
- ✓ **B) If realloc fails and returns NULL, original pointer arr is overwritten with NULL, causing memory leak of original block**
- C) Should use calloc instead
- D) 20*sizeof(int) is invalid

Answer: B

Explanation: If realloc returns NULL (allocation failure), assigning it to arr loses the only pointer to the original block — memory leak. Correct pattern: `int\* tmp = realloc(arr, 20\*sizeof(int)); if (tmp) arr = tmp; else /\* handle error, arr still valid \*/`.

Q161. [Medium]

[C Internals] — Struct Padding — Size Calculation

What is sizeof(S) on a 64-bit system with natural alignment?

```
struct S {
char a; // 1 byte
int b; // 4 bytes
char c; // 1 byte
double d; // 8 bytes
};
```

- A) 14 bytes
- B) 16 bytes
- ✓ **C) 24 bytes**
- D) 20 bytes

Answer: C

Explanation: Layout: char a(1) + 3 padding + int b(4) = 8 bytes. Then char c(1) + 7 padding + double d(8) = 16 bytes. Total: 24. Struct padding aligns each member to its natural alignment and adds trailing padding to make struct size a multiple of its largest member alignment (8).

Q162. [Easy]

[C Internals] — Dangling Pointer — Use After Free

What is the output of this C program?

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int* p = (int*)malloc(sizeof(int));
    *p = 42;
    free(p);
    printf("%d\n", *p); // ?
    return 0;
}
```

- A) 42 always
- B) 0 always
- ✓ C) Undefined behavior — p is a dangling pointer after free
- D) Segfault always

Answer: C

Explanation: After free(p), p points to deallocated memory — a dangling pointer. Dereferencing it is undefined behavior. It may print 42 (value not yet overwritten), 0, garbage, or crash. Correct fix: p = NULL after free.

Q163. [Hard]

[Java Internals] — HashMap Treeification Threshold

In Java 8+, a HashMap bucket converts from a linked list to a Red-Black Tree when:

- A) Load factor > 0.75
- ✓ B) Bucket length exceeds 8 (TREEIFY_THRESHOLD) AND total capacity >= 64
- C) Number of entries > 1000
- D) Hash collision rate > 50%

Answer: B

Explanation: HashMap.TREEIFY_THRESHOLD=8, MIN_TREEIFY_CAPACITY=64. When a bucket's chain exceeds 8 entries AND the total table size is at least 64, the chain is converted to a TreeNode (RB-Tree). Below 64 total capacity, a resize is preferred instead.

Q164. [Medium]

[Java Internals] — String intern() and Memory

What does String.intern() return?

```
String a = new String("hello");
String b = "hello";
String c = a.intern();
System.out.println(b == c); // ?
System.out.println(a == c); // ?
```

- A) false, false
- B) true, true
- ✓ C) true, false
- D) false, true

Answer: C

Explanation: b is an interned string from the string pool. a.intern() returns the canonical representation from the pool — the same object as b. So b==c is true. a itself is a heap object (new String(...)), different from the pool object: a==c is false.

Q165. [Medium]

[Java Internals] — GC Trigger During Deep Recursion

A recursive algorithm allocating a new object at each call depth processes 10^6 levels. The JVM's primary concern is:

- A) CPU overheating
- ✓ B) **StackOverflowError from exhausting the thread stack (each frame ~100-1000 bytes, default stack ~512KB-1MB)**
- C) Heap exhaustion from objects
- D) ClassLoader overhead

Answer: B

Explanation: Default JVM thread stack is ~512KB-1MB. Each stack frame uses memory for local variables and return addresses. At 10^6 levels, even 1 byte per frame overflows. This causes StackOverflowError (not OutOfMemoryError). GC manages heap; the stack is separate.

Q166. [Medium]

[Java Internals] — ArrayList vs LinkedList — Cache Performance

For sequential iteration over 10^6 elements, ArrayList outperforms LinkedList primarily because:

- A) ArrayList has O(1) size() method
- ✓ B) **ArrayList's contiguous memory layout is cache-friendly; LinkedList nodes are scattered in memory causing cache misses**
- C) LinkedList has O(n) get(i)
- D) ArrayList uses primitives internally

Answer: B

Explanation: Modern CPUs prefetch cache lines (64 bytes) sequentially. ArrayList's backing array benefits from spatial locality: iterating through it is cache-friendly. LinkedList nodes are heap-allocated individually with pointers to next nodes — poor spatial locality causes frequent cache misses.

Q167. [Medium]

[Java Internals] — volatile vs synchronized

When should volatile be used instead of synchronized?

- A) When multiple threads update a shared variable
- ✓ **B) When only one thread writes and others read a shared variable (visibility guarantee without atomicity overhead)**
- C) For compound operations like i++
- D) volatile is always faster and should replace synchronized

Answer: B

Explanation: volatile guarantees visibility (writes are immediately flushed to main memory; reads bypass cache). But it does NOT guarantee atomicity. i++ (read-modify-write) requires synchronized or AtomicInteger even with volatile. Use volatile for simple flags written by one thread.

Q168. [Easy]

[Java Internals] — Java Memory Model — Object Creation on Heap

When `new MyObject()` is called in Java, the object is allocated:

- A) On the stack if size < 64 bytes
- ✓ **B) On the heap (Eden space of Young Generation)**
- C) In the Metaspace
- D) In the Old Generation immediately

Answer: B

Explanation: All Java objects are heap-allocated (unless JIT escape analysis proves they don't escape — 'stack allocation' optimization). New objects go to Eden (Young Gen). Long-lived objects are promoted to Old Gen after surviving several GC cycles.

Q169. [Easy]

[C++ Internals] — std::map vs std::unordered_map — Ordered Iteration

Which statement correctly compares `std::map` and `std::unordered_map`?

- A) `unordered_map` always has $O(1)$ lookup
- ✓ **B) `map`: $O(\log n)$ all operations, keys sorted; `unordered_map`: $O(1)$ average / $O(n)$ worst-case, no order**
- C) `map` uses hash table
- D) `unordered_map` supports `lower_bound()`

Answer: B

Explanation: std::map is a Red-Black Tree: $O(\log n)$ for insert/find/erase, keys in sorted order, supports lower_bound/upper_bound. std::unordered_map is a hash table: $O(1)$ average, $O(n)$ worst case, no ordering, no lower_bound.

Q170. [Easy]

[C Internals] — Pointer Arithmetic — Array Navigation

What does the following C code print?

```
int arr[] = {10, 20, 30, 40, 50};  
int* p = arr + 2;  
printf("%d %d\n", *(p-1), *(p+1));
```

- A) 10 40
- ✓ B) 20 40
- C) 30 40
- D) 20 30

Answer: B

*Explanation: p = arr+2 points to arr[2]=30. *(p-1) = arr[1] = 20. *(p+1) = arr[3] = 40. Pointer arithmetic adds multiples of sizeof(int) to the address, not 1 byte.*

Q171. [Hard]

[C++ Internals] — Rule of Three/Five/Zero

C++'s 'Rule of Five' states that if a class defines any of {destructor, copy constructor, copy assignment}, it should also define:

- A) Virtual functions
- ✓ B) Move constructor and move assignment operator (to avoid defaulted moves being incorrect or deleted)
- C) Default constructor
- D) All member functions

Answer: B

Explanation: If you define a destructor (managing resources), the compiler-generated copy/move operations may be wrong. Defining the destructor suppresses default move. The Rule of Five: always define all five (or none and rely on Rule of Zero with RAII wrappers).

Q172. [Hard]

[Java Internals] — ConcurrentHashMap vs HashMap Thread Safety

ConcurrentHashMap in Java 8 achieves thread safety using:

- A) Locking the entire map (like Hashtable)
- ✓ B) Lock-free reads (volatile nodes) and CAS + synchronized on individual buckets for writes
- C) ReadWriteLock on the whole map
- D) Copying the map on each write (copy-on-write)

Answer: B

Explanation: Java 8 ConcurrentHashMap: reads are lock-free using volatile Node references. Writes use CAS for empty buckets and synchronized on the head node of a bucket for non-empty ones. This allows maximum concurrency without locking the whole map.

Q173. [Easy]

[C Internals] — Function Pointer Syntax

Which declaration correctly declares a function pointer to a function taking int, returning double?

- A) double* fp(int)
- B) double (*fp)(int)*
- ✓ C) double (*fp)(int)
- D) (*double)(int) fp

Answer: C

*Explanation: Function pointer syntax: return_type (*pointer_name)(parameter_types). `double (\*fp)(int)` declares fp as a pointer to a function taking int and returning double. Option A declares a function (not pointer) returning double*. Option D is syntactically incorrect.*

Q174. [Easy]

[C++ Internals] — Lambda Capture — Reference vs Value

What does this code print?

```
int x = 10;
auto f = [&x]() { return x * 2; };
x = 20;
std::cout << f();
```

- A) 20
- ✓ B) 40
- C) 0
- D) Undefined behavior

Answer: B

*Explanation: [&x] captures x by reference. When f() is called, x is already 20. So f() returns 20*2 = 40. Contrast with [x] (capture by value at lambda creation time), which would capture x=10 and return 20.*

Q175. [Easy]

[Java Internals] — Integer Cache — == vs equals()

What does this print?

```
Integer a = 127;
Integer b = 127;
Integer c = 128;
Integer d = 128;
System.out.println(a == b); // ?
System.out.println(c == d); // ?
```

- A) false, false
- B) true, true
- ✓ C) true, false
- D) false, true

Answer: C

Explanation: Java caches Integer objects for values -128 to 127. a==b compares the same cached object: true. 128 is outside the cache range: c and d are distinct heap objects, so c==d compares references: false. Always use .equals() for Integer comparison.

Q176. [Medium]

[C++ Internals] — Template Instantiation — Code Bloat

A function template `template void sort(vector&)` instantiated with 10 different types generates:

- A) One function shared by all types
- ✓ B) 10 separate compiled functions (one per type) in the binary
- C) A runtime dispatch table
- D) A single virtual function

Answer: B

Explanation: C++ templates are expanded at compile time. Each unique template instantiation (sort, sort, etc.) generates separate machine code. This can cause 'code bloat' — binary size grows with the number of distinct type instantiations.

Q177. [Easy]

[C Internals] — Stack vs Heap — Return Address Safety

Why is returning a pointer to a local variable dangerous?

```
int* getVal() {  
    int x = 42;  
    return &x; // DANGER?  
}
```

- A) x is const and can't be pointed to
- ✓ B) x is on the stack and is destroyed when getVal() returns; the pointer becomes dangling
- C) This returns the address of x correctly
- D) int* can't point to local variables

Answer: B

Explanation: Local variables live on the stack frame of their function. When getVal() returns, its stack frame is reclaimed. The returned pointer now points to deallocated stack memory — undefined behavior when dereferenced. Return heap-allocated memory or pass a pointer in as a parameter.

Q178. [Easy]

[Java Internals] — Stack Memory — Primitive vs Reference

In Java, when a method executes `int x = 5; String s = new String('hi');`, what is on the stack vs heap?

- A) Both x and s are on the stack
- ✓ B) x (primitive int, value 5) is on the stack; the String object is on the heap; s (reference) is on the stack
- C) Both are on the heap
- D) x is on the heap; s is on the stack

Answer: B

Explanation: Java: primitives and references are stored on the stack frame. The actual object a reference points to is always on the heap. So x=5 (4 bytes on stack), s (reference/pointer, 4-8 bytes on stack), 'hi' String object (heap).

Q179. [Hard]

[C++ Internals] — std::string SSO — Short String Optimization

Most std::string implementations avoid heap allocation for short strings via SSO (Short String Optimization). The typical threshold is:

- A) 4 characters
- B) 8 characters
- ✓ C) 15-22 characters (implementation defined, typically 15 on GCC/Clang, 22 on MSVC)
- D) 64 characters

Answer: C

Explanation: SSO stores short strings directly in the string object's internal buffer (using space that would otherwise store pointer/size/capacity). GCC's libstdc++ uses 15 chars in-place; MSVC uses 15; libc++ uses 22. This avoids heap allocation for common short strings.

Q180. [Easy]

[C Internals] — sizeof Operator — Array Decay

What does this C code print?

```
void printSize(int arr[]) {
    printf("%zu\n", sizeof(arr));
}

int main() {
    int a[10];
    printf("%zu\n", sizeof(a));
    printSize(a);
}
```

- A) 40 and 40
- B) 10 and 10
- ✓ C) 40 and 8 (on 64-bit — array decays to pointer when passed)
- D) 40 and 4

Answer: C

*Explanation: sizeof(a) in main = 10 * sizeof(int) = 40. When passed to a function, arrays decay to pointers: sizeof(arr) inside printSize = sizeof(int*) = 8 (64-bit). This is a classic C pitfall — always pass array size separately.*

Q181. [Medium]

[Java Internals] — Garbage Collection — GC Roots

JVM GC roots include which of the following?

- A) All objects in the Old Generation
- ✓ B) Active thread stacks, static fields, JNI references, and class loader references
- C) Objects with reference count > 0
- D) Objects in the Eden space only

Answer: B

Explanation: GC roots are starting points for reachability analysis. They include: local variables on thread stacks, static fields of loaded classes, active JNI references, and references held by the JVM itself. Any object reachable from a GC root is considered live.

Q182. [Medium]

[C++ Internals] — Undefined Behavior — Signed Integer Overflow

In C++, what is the result of `int x = INT_MAX; x++;`?

- A) Wraps to `INT_MIN` (defined behavior)
- ✓ B) Undefined behavior — signed integer overflow is UB in C++
- C) Throws an exception
- D) Returns 0

Answer: B

Explanation: Unlike unsigned integers (which wrap around modulo 2^n , well-defined in C/C++), signed integer overflow is explicitly undefined behavior. Compilers may assume it never happens and optimize accordingly, producing surprising results if it does occur.

Q183. [Easy]

[Java Internals] — StringBuilder vs String Concatenation

In a loop concatenating n strings with `+: s = s + parts[i]`, the time complexity is:

- A) $O(n)$
- B) $O(n \log n)$
- ✓ C) $O(n^2)$ — each `+` creates a new String object copying previous content
- D) $O(1)$ amortized

Answer: C

Explanation: String is immutable in Java. Each `+` creates a new String, copying all previous content. Concatenating strings of total length L : total work is $1+2+3+\dots+L = O(L^2) = O(n^2)$ for n equal-length strings. Use `StringBuilder` for $O(n)$ concatenation.

Q184. [Hard]

[C Internals] — Union Type Reinterpretation

What does this C code print on a little-endian system?

```
#include <stdio.h>
union U {
    int i;
    char c[4];
};
int main() {
    union U u;
    u.i = 0x01020304;
    printf("%d\n", u.c[0]);
}
```

- A) 1
- B) 3
- ✓ C) 4
- D) Undefined behavior

Answer: C

Explanation: On a little-endian system, the least significant byte is stored first. `0x01020304`: byte 0 (`c[0]`) = `0x04` = 4, byte 1 = `0x03` = 3, byte 2 = `0x02` = 2, byte 3 (most significant) = `0x01` = 1. Answer: 4.

Q185. [Hard]

[C++ Internals] — CRTP — Curiously Recurring Template Pattern

CRTP is used to achieve:

```
template<typename Derived>
class Base {
public:
    void interface() {
        static_cast<Derived*>(this)->implementation();
    }
};

class Derived : public Base<Derived> {
public:
    void implementation() { /* ... */ }
};
```

A) Dynamic polymorphism without overhead

✓ B) Static polymorphism — compile-time dispatch without vtable overhead

C) Multiple inheritance

D) Template specialization

Answer: B

Explanation: CRTP achieves polymorphism resolved at compile time via static_cast, avoiding virtual dispatch overhead (no vtable lookup). Useful for mixins and performance-critical code. The derived class type is known at compile time via the template parameter.

Q186. [Medium]

[Java Internals] — finalize() vs try-with-resources

Why is try-with-resources preferred over finalize() for resource cleanup?

A) finalize() is faster

✓ B) finalize() has non-deterministic timing (depends on GC), may never run, deprecated in Java 9+. try-with-resources guarantees immediate cleanup

C) try-with-resources uses less memory

D) finalize() doesn't handle exceptions

Answer: B

Explanation: finalize() is called by GC — unpredictable timing, possibly never (app may exit), deprecated in Java 9, removed in Java 18. try-with-resources (AutoCloseable.close()) runs deterministically at block exit, guaranteed for resource cleanup.

Q187. [Medium]

[C++ Internals] — std::optional — Avoiding Null Pointer

std::optional has what memory overhead compared to T?

- A) No overhead
- ✓ B) sizeof(T) + 1 byte (minimum, for the 'has value' boolean flag), potentially padded
- C) sizeof(T) * 2
- D) 8 bytes (pointer size) overhead

Answer: B

Explanation: std::optional stores the value in-place (not on the heap) plus a bool indicating presence. Due to alignment, the overhead may be more than 1 byte in practice (padding). This is much more efficient than heap-allocating a pointer.

Q188. [Easy]

[C Internals] — memcpy vs memmove for Overlapping Regions

When source and destination memory regions overlap, which function must be used?

- A) memcpy — it handles overlap
- ✓ B) memmove — it correctly handles overlapping regions by using a temporary buffer or copying in the right direction
- C) strcpy
- D) Either works identically

Answer: B

Explanation: memcpy behavior is undefined for overlapping regions (compiler may use SIMD optimizations assuming no overlap). memmove guarantees correct behavior by checking overlap direction and copying accordingly (back-to-front if necessary to avoid overwriting source data).

Q189. [Easy]

[Java Internals] — Java Generics — Type Erasure

Due to type erasure, List and List at runtime are both:

- A) Different JVM classes
- B) List
- ✓ C) Both just List (raw type) — generic type info is erased at compile time
- D) Integer[] and String[]

Answer: C

Explanation: Java generics use type erasure: generic type parameters are removed at runtime. ArrayList and ArrayList are both ArrayList at the bytecode level. This is why instanceof with generics is not allowed: `obj instanceof List` doesn't compile.

Q190. [Medium]

[C++ Internals] — constexpr vs const — Compile-time Evaluation

What is the key difference between const and constexpr in C++11+?

- A) const is for runtime, constexpr is for compile time
- B) constexpr can only be used with integers
- ✓ C) const guarantees value won't change after initialization; constexpr guarantees evaluation AT COMPILE TIME (enables use in template arguments, array sizes)
- D) They are identical

Answer: C

Explanation: const: value doesn't change after init, but may be initialized at runtime. constexpr: must be computable at compile time. constexpr values can be used as template arguments, array sizes, and in other compile-time contexts. constexpr functions can run at both compile and runtime.

Q191. [Medium]

[Java Internals] — Thread Local Storage in Java

ThreadLocal in Java provides:

- A) Synchronized access to shared T
- B) A lock-free T shared across threads
- ✓ C) A separate instance of T for each thread, stored in Thread's own map
- D) A cached T on the heap

Answer: C

Explanation: ThreadLocal maintains a ThreadLocalMap inside each Thread object. get/set operations on a ThreadLocal access the current thread's own copy — completely isolated from other threads, requiring no synchronization. Useful for thread-safe SimpleDateFormat, database connections, etc.

Q192. [Medium]

[C Internals] — Bit Fields in Structs

How many bytes does this struct likely occupy?

```
struct Flags {  
    unsigned int a : 1;  
    unsigned int b : 3;  
    unsigned int c : 4;  
};
```

- A) 1 byte
- B) 3 bytes
- ✓ C) 4 bytes — packed into one int
- D) 8 bytes

Answer: C

Explanation: Bit fields pack multiple fields into a single storage unit. $a(1)+b(3)+c(4) = 8 \text{ bits} = 1 \text{ byte}$, but the underlying type is unsigned int (4 bytes). The compiler uses one int word for all these fields. $\text{sizeof(Flags)} = 4$.

Q193. [Hard]

[C++ Internals] — Placement New — Custom Allocators

Placement new is used for:

```
char buf[sizeof(MyClass)];  
MyClass* obj = new (buf) MyClass(); // placement new
```

- A) Allocating on the stack automatically
- ✓ B) Constructing an object at a pre-allocated memory address without allocating new memory
- C) Allocating in shared memory
- D) Creating global objects

Answer: B

Explanation: Placement new calls the constructor on an already-allocated memory region (here, the char buffer buf). No memory allocation occurs. Useful for memory pools, shared memory, and embedded systems where allocation is controlled separately from construction.

Q194. [Hard]

[Java Internals] — double-checked Locking — Singleton

The double-checked locking singleton pattern requires volatile because:

```
class Singleton {  
    private static volatile Singleton instance;  
    public static Singleton getInstance() {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null)  
                    instance = new Singleton(); // Why volatile?  
            }  
        }  
        return instance;  
    }  
}
```

- A) synchronized doesn't work without volatile
- ✓ B) Without volatile, CPU/compiler may reorder so a partially constructed instance is visible to other threads before initialization is complete
- C) volatile prevents GC from collecting instance
- D) volatile makes the check faster

Answer: B

Explanation: `new Singleton()` involves: allocate memory, initialize fields, assign reference. Without volatile, JVM/CPU may reorder to: allocate, assign reference, initialize fields. Another thread sees non-null but uninitialized instance. volatile prevents this reordering via memory barrier.

Q195. [Medium]

[C Internals] — getchar Returning int — Not char

Why does getchar() return int instead of char?

```
int c; // NOT char c!
while ((c = getchar()) != EOF) {
    putchar(c);
}
```

- A) For performance reasons
- B) char can't hold letter values
- ✓ C) EOF is -1 (typically), which can't fit in unsigned char (0-255). Using char would make EOF indistinguishable from character 255
- D) getchar returns Unicode code points

Answer: C

Explanation: EOF is typically -1. If c were char (implementation-defined signedness), on unsigned char systems, char can't hold -1 — EOF would wrap to 255, matching the character 'y'. Using int ensures all 256 char values AND EOF (-1) can be distinguished.

Q196. [Easy]

[C++ Internals] — Structured Bindings C++17

What does this C++17 code print?

```
#include <map>
#include <iostream>
int main() {
    std::map<int, std::string> m = {{1, "one"}, {2, "two"}};
    for (auto& [key, val] : m) {
        std::cout << key << ":" << val << " ";
    }
}
```

- A) Compilation error in C++14
- B) 2:two 1:one (unordered)
- ✓ C) 1:one 2:two (map iterates in sorted key order)
- D) Undefined behavior

Answer: C

Explanation: std::map is ordered by key. Structured bindings (C++17) decompose the pair into key and val. Iterating prints keys in ascending order: '1:one 2:two'. This code requires C++17 or later.

Q197. [Hard]

[Java Internals] — String Pool — intern() Memory Impact

Calling intern() on millions of dynamically generated strings risks:

- A) Stack overflow
- ✓ **B) Metaspace/PermGen exhaustion (pre-Java 7: PermGen; Java 7+: heap; Java 8+: native memory via Metaspace)**
- C) Deadlock
- D) ClassCastException

Answer: B

Explanation: The string pool (interning) stores strings in a dedicated area. Pre-Java 7: PermGen (fixed size, easily exhausted). Java 7+: moved to heap (subject to GC). Java 8+: native memory. Interning millions of unique strings still risks memory exhaustion regardless of location.

Q198. [Hard]

[C Internals] — Undefined Behavior — Strict Aliasing

What is the issue with this C code?

```
float f = 3.14f;
int* ip = (int*)&f; // strict aliasing violation?
printf("%d\n", *ip);
```

- A) float cannot be cast to int*
- ✓ **B) Undefined behavior — violates strict aliasing rule; use memcpy or union for type punning**
- C) Prints the integer value 3
- D) Safe with -O0 optimization

Answer: B

Explanation: C's strict aliasing rule: an object may only be accessed through its declared type (or char). Accessing a float through int* violates this, allowing compilers to assume they don't alias and generate incorrect optimized code. Use memcpy(&i, &f, 4) or a union for safe type punning.*

Q199. [Medium]

[C++ Internals] — std::atomic vs mutex for Counter

For a thread-safe counter incremented by multiple threads, std::atomic vs mutex: which is preferred and why?

- A) mutex — always safer
- ✓ **B) std::atomic — uses hardware CAS/lock-prefix instructions; no OS-level locking, lower overhead for single-variable operations**
- C) volatile int — sufficient for threading
- D) They are identical in performance

Answer: B

Explanation: std::atomic uses CPU-level atomic instructions (CMPXCHG, LOCK ADD) without OS involvement — no context switching or kernel calls. mutex involves OS primitives (futex on Linux), much heavier for simple increment. volatile provides no atomicity guarantee.

Q200. [Hard]

[Java Internals] — JVM JIT — Method Inlining Threshold

The JVM's JIT compiler inlines a method when:

- A) The method is declared final
- B) The method is less than 10 bytecodes
- ✓ C) The method is 'hot' (called frequently, typically >10,000 times) and small enough (typically <35 bytecodes for trivial, <325 for complex)
- D) The method has no loops

Answer: C

Explanation: JIT inlining is a key optimization: eliminate call overhead and enable further optimizations (constant folding, escape analysis). HotSpot inlines methods that are 'hot' (profiling threshold) and not too large.

-XX:MaxInlineSize (default 35 bytecodes) and -XX:FreqInlineSize (default 325) control thresholds.

ANSWER KEY

Quick reference for all 200 questions

Q1: B	Q2: B	Q3: B	Q4: C	Q5: A
Q6: C	Q7: B	Q8: B	Q9: C	Q10: B
Q11: C	Q12: B	Q13: A	Q14: B	Q15: B
Q16: C	Q17: B	Q18: B	Q19: B	Q20: B
Q21: B	Q22: C	Q23: A	Q24: A	Q25: C
Q26: A	Q27: B	Q28: B	Q29: B	Q30: C
Q31: A	Q32: B	Q33: B	Q34: B	Q35: C
Q36: B	Q37: B	Q38: B	Q39: B	Q40: C
Q41: A	Q42: B	Q43: B	Q44: B	Q45: D
Q46: B	Q47: B	Q48: B	Q49: C	Q50: C
Q51: C	Q52: B	Q53: B	Q54: B	Q55: B
Q56: B	Q57: C	Q58: A	Q59: B	Q60: C
Q61: B	Q62: B	Q63: C	Q64: B	Q65: B
Q66: C	Q67: A	Q68: B	Q69: C	Q70: B
Q71: B	Q72: B	Q73: C	Q74: B	Q75: B
Q76: C	Q77: B	Q78: B	Q79: C	Q80: B
Q81: C	Q82: A	Q83: B	Q84: B	Q85: B
Q86: C	Q87: B	Q88: C	Q89: C	Q90: C
Q91: B	Q92: C	Q93: C	Q94: B	Q95: C
Q96: B	Q97: B	Q98: C	Q99: C	Q100: B
Q101: C	Q102: A	Q103: B	Q104: B	Q105: B
Q106: B	Q107: B	Q108: B	Q109: B	Q110: B
Q111: B	Q112: A	Q113: C	Q114: B	Q115: B
Q116: B	Q117: B	Q118: B	Q119: B	Q120: C
Q121: B	Q122: C	Q123: B	Q124: B	Q125: B
Q126: B	Q127: B	Q128: B	Q129: B	Q130: B
Q131: C	Q132: B	Q133: B	Q134: B	Q135: B
Q136: B	Q137: C	Q138: C	Q139: C	Q140: B
Q141: B	Q142: A	Q143: C	Q144: B	Q145: C
Q146: B	Q147: B	Q148: C	Q149: C	Q150: B
Q151: C	Q152: A	Q153: B	Q154: A	Q155: B
Q156: B	Q157: C	Q158: B	Q159: B	Q160: B
Q161: C	Q162: C	Q163: B	Q164: C	Q165: B
Q166: B	Q167: B	Q168: B	Q169: B	Q170: B

Q171: B	Q172: B	Q173: C	Q174: B	Q175: C
Q176: B	Q177: B	Q178: B	Q179: C	Q180: C
Q181: B	Q182: B	Q183: C	Q184: C	Q185: B
Q186: B	Q187: B	Q188: B	Q189: C	Q190: C
Q191: C	Q192: C	Q193: B	Q194: B	Q195: C
Q196: C	Q197: B	Q198: B	Q199: B	Q200: C

Legend: ✓ indicates correct answer